

Business Case: Delhivery - Feature Engineering



About Delhivery

Delhivery is the largest and fastest-growing fully integrated player in India by revenue in Fiscal 2021. They aim to build the operating system for commerce, through a combination of world-class infrastructure, logistics operations of the highest quality, and cutting-edge engineering and technology capabilities.

The Data team builds intelligence and capabilities using this data that helps them to widen the gap between the quality, efficiency, and profitability of their business versus their competitors.

The company wants to understand and process the data coming out of data engineering pipelines:

- Clean, sanitize and manipulate data to get useful features out of raw fields
- Make sense out of the raw data and help the data science team to build forecasting models on it

Importing Required Libraries

```
In [ ]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib as mpl
import matplotlib.pyplot as plt
import scipy.stats as spy
```

```
In [ ]: import warnings
warnings.simplefilter('ignore')
```

Loading the Dataset

```
In [ ]: df=pd.read_csv("https://d2beiqlkhq929f0.cloudfront.net/public_assets/assets/000/001/551/original/delhivery_data.csv?1642751181")
df
```

Out []:

	data	trip_creation_time	route_schedule_uuid	route_type	trip_uuid	source_center	source_name	destinati
0	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	153741093647649320	IND388121AAA	Anand_VUNagar_DC (Gujarat)	IND3
1	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	153741093647649320	IND388121AAA	Anand_VUNagar_DC (Gujarat)	IND3
2	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	153741093647649320	IND388121AAA	Anand_VUNagar_DC (Gujarat)	IND3
3	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	153741093647649320	IND388121AAA	Anand_VUNagar_DC (Gujarat)	IND3
4	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	153741093647649320	IND388121AAA	Anand_VUNagar_DC (Gujarat)	IND3
...	...	...	...	...	...	...	...	...
144862	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	153746066843555182	IND131028AAB	Sonipat_Kundli_H (Haryana)	IND0
144863	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	153746066843555182	IND131028AAB	Sonipat_Kundli_H (Haryana)	IND0
144864	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	153746066843555182	IND131028AAB	Sonipat_Kundli_H (Haryana)	IND0
144865	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	153746066843555182	IND131028AAB	Sonipat_Kundli_H (Haryana)	IND0
144866	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	153746066843555182	IND131028AAB	Sonipat_Kundli_H (Haryana)	IND0

144867 rows × 24 columns



--Dropping unwanted columns--

```
In [ ]: df.drop( columns=['is_cutoff','cutoff_factor','cutoff_timestamp','factor','segment_factor'],inplace=True)
```

--Non graphical exploratory analysis--

```
In [ ]: print(f" Rows: {df.shape[0]} \n Columns: {df.shape[1]}")
```

Rows: 144867
Columns: 19

```
In [ ]: print(f"Column name are {df.columns}")
```

Column name are Index(['data', 'trip_creation_time', 'route_schedule_uuid', 'route_type', 'trip_uuid', 'source_center', 'source_name', 'destination_center', 'destination_name', 'od_start_time', 'od_end_time', 'start_scan_to_end_scan', 'actual_distance_to_destination', 'actual_time', 'osrm_time', 'osrm_distance', 'segment_actual_time', 'segment_osrm_time', 'segment_osrm_distance'], dtype='object')

```
In [ ]: df.dtypes
```

Out[]:

		0
	data	object
	trip_creation_time	object
	route_schedule_uuid	object
	route_type	object
	trip_uuid	object
	source_center	object
	source_name	object
	destination_center	object
	destination_name	object
	od_start_time	object
	od_end_time	object
	start_scan_to_end_scan	float64
	actual_distance_to_destination	float64
	actual_time	float64
	osrm_time	float64
	osrm_distance	float64
	segment_actual_time	float64
	segment_osrm_time	float64
	segment_osrm_distance	float64

dtype: object

```
In [ ]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 144867 entries, 0 to 144866
Data columns (total 19 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   data                                  144867 non-null object
1   trip_creation_time                    144867 non-null object
2   route_schedule_uuid                  144867 non-null object
3   route_type                           144867 non-null object
4   trip_uuid                            144867 non-null object
5   source_center                        144867 non-null object
6   source_name                          144574 non-null object
7   destination_center                   144867 non-null object
8   destination_name                     144606 non-null object
9   od_start_time                        144867 non-null object
10  od_end_time                          144867 non-null object
11  start_scan_to_end_scan                144867 non-null float64
12  actual_distance_to_destination         144867 non-null float64
13  actual_time                           144867 non-null float64
14  osrm_time                             144867 non-null float64
15  osrm_distance                         144867 non-null float64
16  segment_actual_time                   144867 non-null float64
17  segment_osrm_time                     144867 non-null float64
18  segment_osrm_distance                 144867 non-null float64
dtypes: float64(8), object(11)
memory usage: 21.0+ MB
```

```
In [ ]: df.describe()
```

Out[]:

	start_scan_to_end_scan	actual_distance_to_destination	actual_time	osrm_time	osrm_distance	segment_actual_time	segment_
count	144867.000000	144867.000000	144867.000000	144867.000000	144867.000000	144867.000000	1448
mean	961.262986	234.073372	416.927527	213.868272	284.771297	36.196111	
std	1037.012769	344.990009	598.103621	308.011085	421.119294	53.571158	
min	20.000000	9.000045	9.000000	6.000000	9.008200	-244.000000	
25%	161.000000	23.355874	51.000000	27.000000	29.914700	20.000000	
50%	449.000000	66.126571	132.000000	64.000000	78.525800	29.000000	
75%	1634.000000	286.708875	513.000000	257.000000	343.193250	40.000000	
max	7898.000000	1927.447705	4532.000000	1686.000000	2326.199100	3051.000000	16

Handle missing values in the data :

In []:

np.any(df.isnull())

Out[]:

np.True_

In []:

df.isnull().sum()

Out[]:

	0
data	0
trip_creation_time	0
route_schedule_uuid	0
route_type	0
trip_uuid	0
source_center	0
source_name	293
destination_center	0
destination_name	261
od_start_time	0
od_end_time	0
start_scan_to_end_scan	0
actual_distance_to_destination	0
actual_time	0
osrm_time	0
osrm_distance	0
segment_actual_time	0
segment_osrm_time	0
segment_osrm_distance	0

dtype: int64

--Filling null values--

In []:

df.fillna(value={'destination_name': 'mode'},inplace=True)
df.fillna(value={'source_name': 'mode'},inplace=True)

In []:

category_columns=df.dtypes=="object"
numerical_columns=df.dtypes!="object"

In []:

unique values in each column
for i in df.columns[category_columns]:
 print(i,":",df[i].nunique())

data : 2
trip_creation_time : 14817
route_schedule_uuid : 1504
route_type : 2
trip_uuid : 14817
source_center : 1508
source_name : 1499
destination_center : 1481
destination_name : 1469
od_start_time : 26369
od_end_time : 26369

```
In [ ]: # unique count of numerical columns
        for i in df.columns[numerical_columns]:
            print(i,":",df[i].nunique())
```

start_scan_to_end_scan : 1915
actual_distance_to_destination : 144515
actual_time : 3182
osrm_time : 1531
osrm_distance : 138046
segment_actual_time : 747
segment_osrm_time : 214
segment_osrm_distance : 113799

```
In [ ]: category_columns[category_columns]
```

Out []:

0	
data	True
trip_creation_time	True
route_schedule_uuid	True
route_type	True
trip_uuid	True
source_center	True
source_name	True
destination_center	True
destination_name	True
od_start_time	True
od_end_time	True

dtype: bool

```
In [ ]: numerical_columns[numerical_columns]
```

Out []:

0	
start_scan_to_end_scan	True
actual_distance_to_destination	True
actual_time	True
osrm_time	True
osrm_distance	True
segment_actual_time	True
segment_osrm_time	True
segment_osrm_distance	True

dtype: bool

--Convert object type To DateTime--

```
In [ ]: df['trip_creation_time']=pd.to_datetime(df['trip_creation_time'])
        df['od_start_time']=pd.to_datetime(df['od_start_time'])
        df['od_end_time']=pd.to_datetime(df['od_end_time'])
```

```
In [ ]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 144867 entries, 0 to 144866
Data columns (total 19 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   data                                  144867 non-null object
1   trip_creation_time                   144867 non-null datetime64[ns]
2   route_schedule_uuid                 144867 non-null object
3   route_type                           144867 non-null object
4   trip_uuid                            144867 non-null object
5   source_center                        144867 non-null object
6   source_name                          144867 non-null object
7   destination_center                  144867 non-null object
8   destination_name                     144867 non-null object
9   od_start_time                       144867 non-null datetime64[ns]
10  od_end_time                           144867 non-null datetime64[ns]
11  start_scan_to_end_scan               144867 non-null float64
12  actual_distance_to_destination       144867 non-null float64
13  actual_time                           144867 non-null float64
14  osrm_time                            144867 non-null float64
15  osrm_distance                        144867 non-null float64
16  segment_actual_time                  144867 non-null float64
17  segment_osrm_time                    144867 non-null float64
18  segment_osrm_distance                144867 non-null float64
dtypes: datetime64[ns](3), float64(8), object(8)
memory usage: 21.0+ MB
```

--Merging of rows and aggregation of fields --

```
In [ ]: grouping_1 = ['trip_uuid', 'source_center', 'destination_center']
df1 = df.groupby(by = grouping_1, as_index = False).agg({'data' : 'first',
                                                         'route_type' : 'first',
                                                         'trip_creation_time' : 'first',
                                                         'source_name' : 'first',
                                                         'destination_name' : 'last',
                                                         'od_start_time' : 'first',
                                                         'od_end_time' : 'first',
                                                         'start_scan_to_end_scan' : 'first',
                                                         'actual_distance_to_destination' : 'last',
                                                         'actual_time' : 'last',
                                                         'osrm_time' : 'last',
                                                         'osrm_distance' : 'last',
                                                         'segment_actual_time' : 'sum',
                                                         'segment_osrm_time' : 'sum',
                                                         'segment_osrm_distance' : 'sum'})
```

```
In [ ]: df1
```

Out []:	trip_uuid	source_center	destination_center	data	route_type	trip_creation_time	source_name	des
0	trip-153671041653548748	IND209304AAA	IND000000ACB	training	FTL	2018-09-12 00:00:16.535741	Kanpur_Central_H_6 (Uttar Pradesh)	Gurgaon
1	trip-153671041653548748	IND462022AAA	IND209304AAA	training	FTL	2018-09-12 00:00:16.535741	Bhopal_Trnsport_H (Madhya Pradesh)	Kanpur
2	trip-153671042288605164	IND561203AAB	IND562101AAA	training	Carting	2018-09-12 00:00:22.886430	Doddablpur_ChikaDPP_D (Karnataka)	Chikblal
3	trip-153671042288605164	IND572101AAA	IND561203AAB	training	Carting	2018-09-12 00:00:22.886430	Tumkur_Veersagr_I (Karnataka)	Doddablp
4	trip-153671043369099517	IND000000ACB	IND160002AAC	training	FTL	2018-09-12 00:00:33.691250	Gurgaon_Bilaspur_HB (Haryana)	Chandigarh
...	...	...	...	...	...	...	...	...
26363	trip-153861115439069069	IND628204AAA	IND627657AAA	test	Carting	2018-10-03 23:59:14.390954	Tirchchndr_Shnmgprm_D (Tamil Nadu)	Thisayanvil
26364	trip-153861115439069069	IND628613AAA	IND627005AAA	test	Carting	2018-10-03 23:59:14.390954	Peikulam_SriVnktpm_D (Tamil Nadu)	Tirunel
26365	trip-153861115439069069	IND628801AAA	IND628204AAA	test	Carting	2018-10-03 23:59:14.390954	Eral_Busstand_D (Tamil Nadu)	Tirchchndr
26366	trip-153861118270144424	IND583119AAA	IND583101AAA	test	FTL	2018-10-03 23:59:42.701692	Sandur_WrdN1DPP_D (Karnataka)	Bellary.
26367	trip-153861118270144424	IND583201AAA	IND583119AAA	test	FTL	2018-10-03 23:59:42.701692	Hospet (Karnataka)	Sandur

26368 rows × 18 columns



Calculate the time taken between od_start_time and od_end_time and keep it as a feature. Drop the original columns, if required :

```
In [ ]: df1['od_total_time'] = df1['od_end_time'] - df1['od_start_time']
df1.drop(columns = ['od_end_time', 'od_start_time'], inplace = True)
df1['od_total_time'] = df1['od_total_time'].apply(lambda x : round(x.total_seconds() / 60.0, 2))
df1['od_total_time'].head()
```

Out[]:

	od_total_time
0	1260.60
1	999.51
2	58.83
3	122.78
4	834.64

dtype: float64

```
In [ ]: df2 = df1.groupby(by = 'trip_uuid', as_index = False).agg({'source_center' : 'first',
                                                                    'destination_center' : 'last',
                                                                    'data' : 'first',
                                                                    'route_type' : 'first',
                                                                    'trip_creation_time' : 'first',
                                                                    'source_name' : 'first',
                                                                    'destination_name' : 'last',
                                                                    'od_total_time' : 'sum',
                                                                    'start_scan_to_end_scan' : 'sum',
                                                                    'actual_distance_to_destination' : 'sum',
                                                                    'actual_time' : 'sum',
                                                                    'osrm_time' : 'sum',
                                                                    'osrm_distance' : 'sum',
                                                                    'segment_actual_time' : 'sum',
                                                                    'segment_osrm_time' : 'sum',
                                                                    'segment_osrm_distance' : 'sum'})
```

```
In [ ]: df2
```

Out[]:

	trip_uuid	source_center	destination_center	data	route_type	trip_creation_time	source_name	de
0	trip-153671041653548748	IND209304AAA	IND209304AAA	training	FTL	2018-09-12 00:00:16.535741	Kanpur_Central_H_6 (Uttar Pradesh)	Kan
1	trip-153671042288605164	IND561203AAB	IND561203AAB	training	Carting	2018-09-12 00:00:22.886430	Doddablpur_ChikaDPP_D (Karnataka)	Doddablj
2	trip-153671043369099517	IND000000ACB	IND000000ACB	training	FTL	2018-09-12 00:00:33.691250	Gurgaon_Bilaspur_HB (Haryana)	Gurgu
3	trip-153671046011330457	IND400072AAB	IND401104AAA	training	Carting	2018-09-12 00:01:00.113710	Mumbai Hub (Maharashtra)	Mu
4	trip-153671052974046625	IND583101AAA	IND583119AAA	training	FTL	2018-09-12 00:02:09.740725	Bellary_Dc (Karnataka)	Sandu
...	...	...	...	...	...	...	...	...
14812	trip-153861095625827784	IND160002AAC	IND160002AAC	test	Carting	2018-10-03 23:55:56.258533	Chandigarh_Mehmdpur_H (Punjab)	Chandigar
14813	trip-153861104386292051	IND121004AAB	IND121004AAA	test	Carting	2018-10-03 23:57:23.863155	FBD_Balabhgarh_DPC (Haryana)	Faridal
14814	trip-153861106442901555	IND208006AAA	IND208006AAA	test	Carting	2018-10-03 23:57:44.429324	Kanpur_GovndNgr_DC (Uttar Pradesh)	Kanpu
14815	trip-153861115439069069	IND627005AAA	IND628204AAA	test	Carting	2018-10-03 23:59:14.390954	Tirunelveli_VdkkuSrt_I (Tamil Nadu)	Tirchchno
14816	trip-153861118270144424	IND583119AAA	IND583119AAA	test	FTL	2018-10-03 23:59:42.701692	Sandur_WrdN1DPP_D (Karnataka)	Sandu

14817 rows × 17 columns



Build some features to prepare the data for actual analysis. Extract features from the below fields:

Source Name: Split and extract features out of destination. City-place-code (State) :


```
In [ ]: def location_name_to_state(x):
        l = x.split('(')
        if len(l) == 1:
            return l[0]
        else:
            return l[1].replace(')', '')
```

```
In [ ]: def location_name_to_city(x):
        if 'location' in x:
            return 'unknown_city'
        else:
            l = x.split()[0].split('_')
            if 'CCU' in x:
                return 'Kolkata'
            elif 'MAA' in x.upper():
                return 'Chennai'
            elif ('HBR' in x.upper()) or ('BLR' in x.upper()):
                return 'Bengaluru'
            elif 'FBD' in x.upper():
                return 'Faridabad'
            elif 'BOM' in x.upper():
                return 'Mumbai'
            elif 'DEL' in x.upper():
                return 'Delhi'
            elif 'OK' in x.upper():
                return 'Delhi'
            elif 'GZB' in x.upper():
                return 'Ghaziabad'
            elif 'GGN' in x.upper():
                return 'Gurgaon'
            elif 'AMD' in x.upper():
                return 'Ahmedabad'
            elif 'CJB' in x.upper():
                return 'Coimbatore'
            elif 'HYD' in x.upper():
                return 'Hyderabad'
            return l[0]
```

```
In [ ]: def location_name_to_place(x):
        if 'location' in x:
            return x
        elif 'HBR' in x:
            return 'HBR Layout PC'
        else:
            l = x.split()[0].split('_', 1)
            if len(l) == 1:
                return 'unknown_place'
            else:
                return l[1]
```

```
In [ ]: df2['source_state'] = df2['source_name'].apply(location_name_to_state)
df2['source_state'].unique()
```

```
Out[ ]: array(['Uttar Pradesh', 'Karnataka', 'Haryana', 'Maharashtra',
               'Tamil Nadu', 'Gujarat', 'Delhi', 'Telangana', 'Rajasthan',
               'Assam', 'Madhya Pradesh', 'West Bengal', 'Andhra Pradesh',
               'Punjab', 'Chandigarh', 'Goa', 'Jharkhand', 'Pondicherry',
               'Orissa', 'Uttarakhand', 'Himachal Pradesh', 'Kerala',
               'Arunachal Pradesh', 'Bihar', 'Chhattisgarh',
               'Dadra and Nagar Haveli', 'Jammu & Kashmir', 'Mizoram', 'Nagaland',
               'mode'], dtype=object)
```

```
In [ ]: df2['source_city'] = df2['source_name'].apply(location_name_to_city)
print('No of source cities :', df2['source_city'].nunique())
df2['source_city'].unique()[:100]
```

No of source cities : 690

```
Out[ ]: array(['Kanpur', 'Doddablpur', 'Gurgaon', 'Mumbai', 'Bellary', 'Chennai',
               'Bengaluru', 'Surat', 'Delhi', 'Pune', 'Faridabad', 'Shirala',
               'Hyderabad', 'Thirumalagiri', 'Gulbarga', 'Jaipur', 'Allahabad',
               'Guwahati', 'Narsinghpur', 'Shrirampur', 'Madakasira', 'Sonari',
               'Dindigul', 'Jalandhar', 'Chandigarh', 'Deoli', 'Pandharpur',
               'Kolkata', 'Bhandara', 'Kurnool', 'Bhiwandi', 'Bhatinda',
               'RoopNagar', 'Bantwal', 'Lalru', 'Kadi', 'Shahdol', 'Gangakher',
               'Durgapur', 'Vapi', 'Jamjodhpur', 'Jetpur', 'Mehsana', 'Jabalpur',
               'Junagadh', 'Gundlupet', 'Mysore', 'Goa', 'Bhopal', 'Sonipat',
               'Himmatnagar', 'Jamshedpur', 'Pondicherry', 'Anand', 'Udgir',
               'Nadiad', 'Villupuram', 'Purulia', 'Bhubaneshwar', 'Bamangola',
               'Tiruppattur', 'Kotdwara', 'Medak', 'Bangalore', 'Dhrangadhra',
               'Hospet', 'Ghumarwin', 'Agra', 'Sitapur', 'Canacona', 'Bilimora',
               'SultnBthry', 'Lucknow', 'Vellore', 'Bhuj', 'Dinhata',
               'Margherita', 'Boisar', 'Vizag', 'Tezpur', 'Koduru', 'Tirupati',
               'Pen', 'Ahmedabad', 'Faizabad', 'Gandhinagar', 'Anantapur',
               'Betul', 'Panskura', 'Rasipuram', 'Sankari', 'Jorhat', 'PNQ',
               'Srikakulam', 'Dehradun', 'Jassur', 'Sawantwadi', 'Shajapur',
               'Ludhiana', 'GreaterThane'], dtype=object)
```



```
In [ ]: df2['source_place'] = df2['source_name'].apply(location_name_to_place)
df2['source_place'].unique()[:100]
```

```
Out[ ]: array(['Central_H_6', 'ChikaDPP_D', 'Bilaspur_HB', 'unknown_place', 'Dc',
              'Poonamallee', 'Chrompet_DPC', 'HBR Layout PC', 'Central_D_12',
              'Lajpat_IP', 'North_D_3', 'Balabhgarh_DPC', 'Central_DPP_3',
              'Shamshbd_H', 'Xroad_D', 'Nehrugn_I', 'Central_I_7',
              'Central_H_1', 'Nangli_IP', 'North', 'KndliDPP_D', 'Central_D_9',
              'DavkharRd_D', 'Bandel_D', 'RTCStand_D', 'Central_DPP_1',
              'KGAirprt_HB', 'North_D_2', 'Central_D_1', 'DC', 'Mthurard_L',
              'Mullanpr_DC', 'Central_DPP_2', 'RajCmplx_D', 'Beliaghata_DPC',
              'RjnaidPP_D', 'AbbasNgr_I', 'Mankoli_HB', 'DPC', 'Airport_H',
              'Hub', 'Gateway_HB', 'Tathawde_H', 'ChotiHv1_DC', 'Trmltmp1_D',
              'OnkarDPP_D', 'Mehmdpur_H', 'KaranNGR_D', 'Sohagpur_D',
              'Chrompet_L', 'Busstand_D', 'Central_I_1', 'IndEstat_I', 'Court_D',
              'Panchot_IP', 'Adhartal_IP', 'DumDum_DPC', 'Bomsndra_HB',
              'Swamylyt_D', 'Yadvigiri_IP', 'Old', 'Kundli_H', 'Central_I_3',
              'Vasanthm_I', 'Poonamallee_HB', 'VUNagar_DC', 'NlgaonRd_D',
              'Bnnrgha_L', 'Thirumtr_IP', 'GariDPP_D', 'Jogshwri_I',
              'KoilStrt_D', 'CotnGren_M', 'Nzbadrd_D', 'Dwaraka_D', 'Nelmngla_H',
              'NvygRDPP_D', 'Gndhichk_D', 'Central_D_3', 'Chowk_D', 'CharRsta_D',
              'Kollgpra_D', 'Peenya_IP', 'GndhiNgr_IP', 'Sanpada_I',
              'WrdN4DPP_D', 'Sakinaka_RP', 'CivilHPL_D', 'OstwlEmp_D',
              'Gajuwaka', 'Mhbhirab_D', 'MGRoad_D', 'Balajicly_I', 'BljiMrkt_D',
              'Dankuni_HB', 'Trnsport_H', 'Rakhial', 'Memnagar', 'East_I_21',
              'Mithakal_D'], dtype=object)
```

Destination Name: Split and extract features out of destination. City-place-code (State) :

```
In [ ]: df2['destination_state'] = df2['destination_name'].apply(location_name_to_state)
df2['destination_state'].head(10)
```

Out[]:

	destination_state
0	Uttar Pradesh
1	Karnataka
2	Haryana
3	Maharashtra
4	Karnataka
5	Tamil Nadu
6	Tamil Nadu
7	Karnataka
8	Gujarat
9	Delhi

dtype: object

```
In [ ]: df2['destination_city'] = df2['destination_name'].apply(location_name_to_city)
df2['destination_city'].head()
```

Out[]:

	destination_city
0	Kanpur
1	Doddablpur
2	Gurgaon
3	Mumbai
4	Sandur

dtype: object

```
In [ ]: df2['destination_place'] = df2['destination_name'].apply(location_name_to_place)
df2['destination_place'].head()
```

```
Out[ ]: 

|   | destination_place |
|---|-------------------|
| 0 | Central_H_6       |
| 1 | ChikaDPP_D        |
| 2 | Bilaspur_HB       |
| 3 | MiraRd_IP         |
| 4 | WrdN1DPP_D        |


```

dtype: object

Trip_creation_time: Extract features like month, year and day :

```
In [ ]: df2['trip_creation_date'] = pd.to_datetime(df2['trip_creation_time'].dt.date)
df2['trip_creation_date'].head()
```

```
Out[ ]: 

|   | trip_creation_date |
|---|--------------------|
| 0 | 2018-09-12         |
| 1 | 2018-09-12         |
| 2 | 2018-09-12         |
| 3 | 2018-09-12         |
| 4 | 2018-09-12         |


```

dtype: datetime64[ns]

```
In [ ]: df2['trip_creation_day'] = df2['trip_creation_time'].dt.day
df2['trip_creation_day'] = df2['trip_creation_day'].astype('int8')
df2['trip_creation_day'].head()
```

```
Out[ ]: 

|   | trip_creation_day |
|---|-------------------|
| 0 | 12                |
| 1 | 12                |
| 2 | 12                |
| 3 | 12                |
| 4 | 12                |


```

dtype: int8

```
In [ ]: df2['trip_creation_month'] = df2['trip_creation_time'].dt.month
df2['trip_creation_month'] = df2['trip_creation_month'].astype('int8')
df2['trip_creation_month'].head()
```

```
Out[ ]: 

|   | trip_creation_month |
|---|---------------------|
| 0 | 9                   |
| 1 | 9                   |
| 2 | 9                   |
| 3 | 9                   |
| 4 | 9                   |


```

dtype: int8

```
In [ ]: df2['trip_creation_year'] = df2['trip_creation_time'].dt.year
df2['trip_creation_year'] = df2['trip_creation_year'].astype('int16')
df2['trip_creation_year'].head()
```

```
Out[ ]:      trip_creation_year
0          2018
1          2018
2          2018
3          2018
4          2018
```

dtype: int16

```
In [ ]: df2['trip_creation_week'] = df2['trip_creation_time'].dt.isocalendar().week
df2['trip_creation_week'] = df2['trip_creation_week'].astype('int8')
df2['trip_creation_week'].head()
```

```
Out[ ]:      trip_creation_week
0          37
1          37
2          37
3          37
4          37
```

dtype: int8

```
In [ ]: df2['trip_creation_hour'] = df2['trip_creation_time'].dt.hour
df2['trip_creation_hour'] = df2['trip_creation_hour'].astype('int8')
df2['trip_creation_hour'].head()
```

```
Out[ ]:      trip_creation_hour
0          0
1          0
2          0
3          0
4          0
```

dtype: int8

Finding the structure of data after data cleaning :

```
In [ ]: df2.shape
```

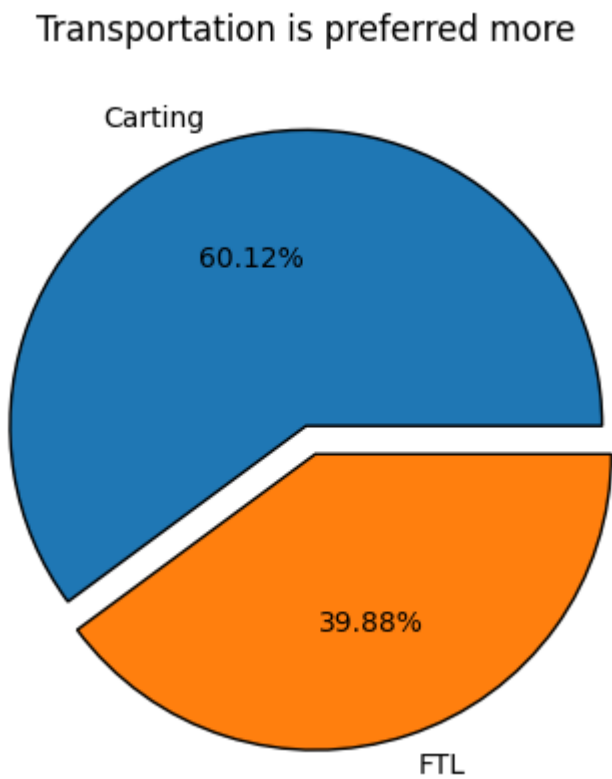
```
Out[ ]: (14817, 29)
```

```
In [ ]: df2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14817 entries, 0 to 14816
Data columns (total 29 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   trip_uuid                                14817 non-null   object
1   source_center                            14817 non-null   object
2   destination_center                       14817 non-null   object
3   data                                     14817 non-null   object
4   route_type                              14817 non-null   object
5   trip_creation_time                       14817 non-null   datetime64[ns]
6   source_name                              14817 non-null   object
7   destination_name                         14817 non-null   object
8   od_total_time                            14817 non-null   float64
9   start_scan_to_end_scan                  14817 non-null   float64
10  actual_distance_to_destination            14817 non-null   float64
11  actual_time                              14817 non-null   float64
12  osrm_time                                14817 non-null   float64
13  osrm_distance                            14817 non-null   float64
14  segment_actual_time                      14817 non-null   float64
15  segment_osrm_time                        14817 non-null   float64
16  segment_osrm_distance                    14817 non-null   float64
17  source_state                             14817 non-null   object
18  source_city                             14817 non-null   object
19  source_place                             14817 non-null   object
20  destination_state                        14817 non-null   object
21  destination_city                         14817 non-null   object
22  destination_place                        14817 non-null   object
23  trip_creation_date                       14817 non-null   datetime64[ns]
24  trip_creation_day                        14817 non-null   int8
25  trip_creation_month                      14817 non-null   int8
26  trip_creation_year                       14817 non-null   int16
27  trip_creation_week                       14817 non-null   int8
28  trip_creation_hour                       14817 non-null   int8
dtypes: datetime64[ns](2), float64(9), int16(1), int8(4), object(13)
memory usage: 2.8+ MB
```

which transportation is preferred more:

```
In [ ]: plt.pie(df2['route_type'].value_counts(),labels=df2['route_type'].value_counts().index,autopct='%1.2f%',wedgeprops={'edgecolor': 'black'})
plt.title('Transportation is preferred more')
plt.show()
```



Most of them using carting shipments rout type.
FTL:39.88%
Carting:60.12%

Trips created on hourly basis :

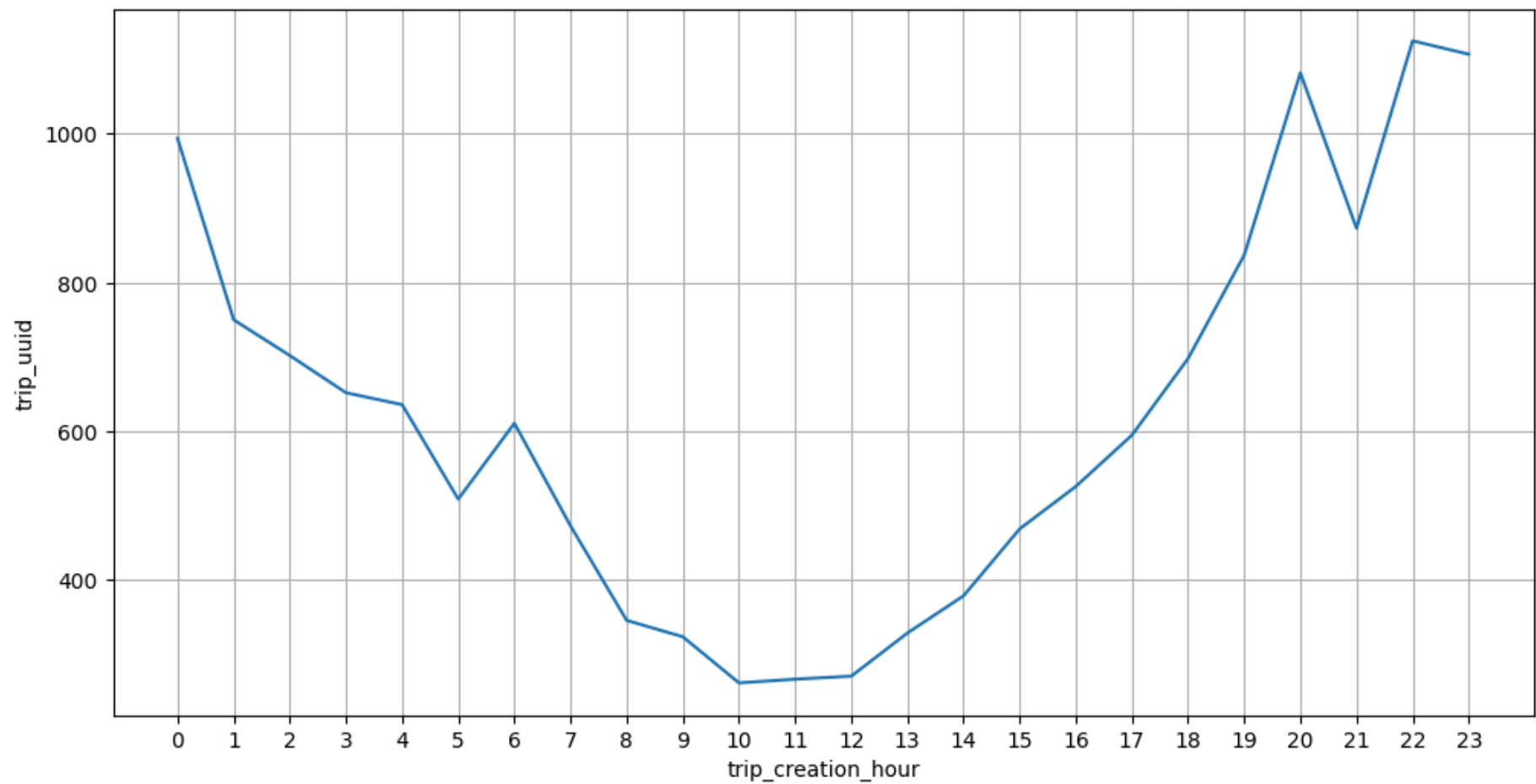
```
In [ ]: df_hour = df2.groupby(by = 'trip_creation_hour')['trip_uuid'].count().to_frame().reset_index()
df_hour
```

Out[]:

	trip_creation_hour	trip_uuid
0	0	994
1	1	750
2	2	702
3	3	652
4	4	636
5	5	509
6	6	611
7	7	473
8	8	346
9	9	324
10	10	262
11	11	267
12	12	271
13	13	329
14	14	379
15	15	469
16	16	526
17	17	595
18	18	698
19	19	837
20	20	1082
21	21	873
22	22	1125
23	23	1107

```
In [ ]: plt.figure(figsize = (12, 6))
sns.lineplot(data = df_hour,
             x = df_hour['trip_creation_hour'],
             y = df_hour['trip_uuid'],
             markers = '*')
plt.xticks(np.arange(0,24))
plt.grid('both')
plt.plot()
```

Out[]: []



- Inference from the above plot that, The number of trips start increasing after the noon from 12 PM, becomes maximum at 10 PM and then start decreasing.

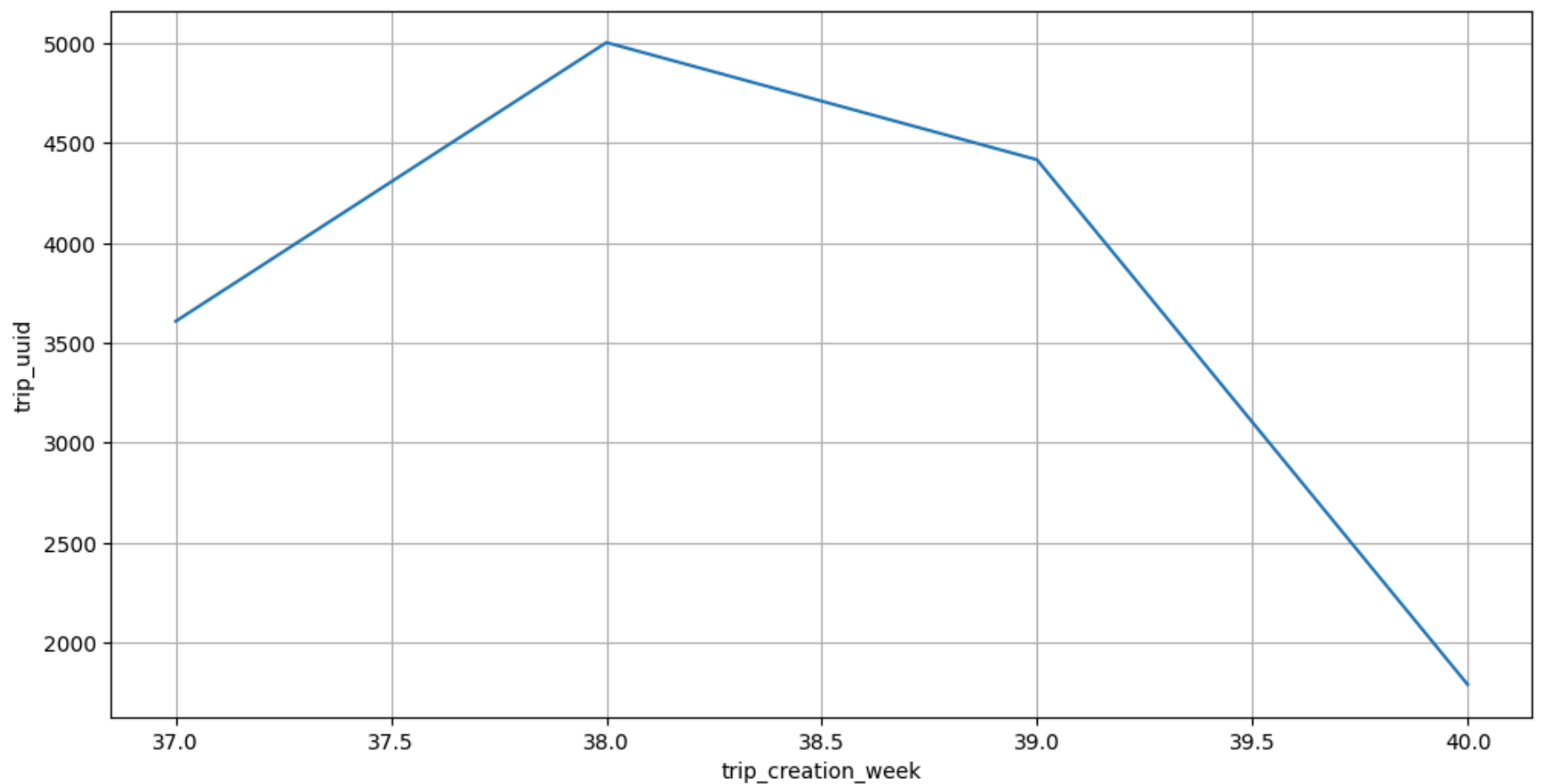
Trips created for different weeks :

```
In [ ]: df_week = df2.groupby(by = 'trip_creation_week')['trip_uuid'].count().to_frame().reset_index()
df_week
```

```
Out[ ]:   trip_creation_week  trip_uuid
0                37        3608
1                38        5004
2                39        4417
3                40        1788
```

```
In [ ]: plt.figure(figsize = (12, 6))
sns.lineplot(data = df_week,
             x = df_week['trip_creation_week'],
             y = df_week['trip_uuid'],
             markers = 'o')
plt.grid('both')
plt.plot()
```

```
Out[ ]: []
```



- It can be inferred from the above plot that most of the trips are created in the 38th week.

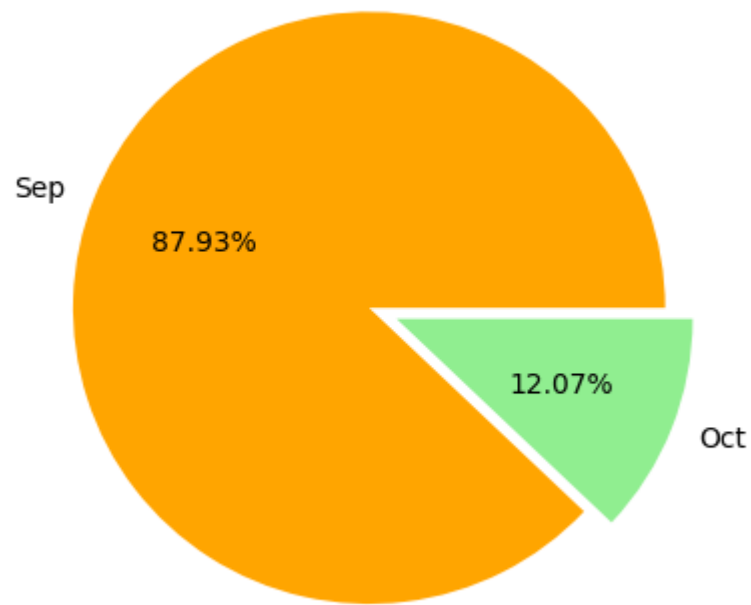
Trips created in given 2 months :

```
In [ ]: df_month = df2.groupby(by = 'trip_creation_month')['trip_uuid'].count().to_frame().reset_index()
df_month['perc'] = np.round(df_month['trip_uuid'] * 100 / df_month['trip_uuid'].sum(), 2)
df_month.head()
```

```
Out[ ]:   trip_creation_month  trip_uuid  perc
0                9        13029  87.93
1               10         1788  12.07
```

```
In [ ]: plt.pie(x = df_month['trip_uuid'],
               labels = ['Sep', 'Oct'],
               explode = [0, 0.1],
               autopct = '%.2f%%',
               colors = ['orange', 'lightgreen'])
plt.plot()
```

```
Out[ ]: []
```

Distribution of trip data for the orders :

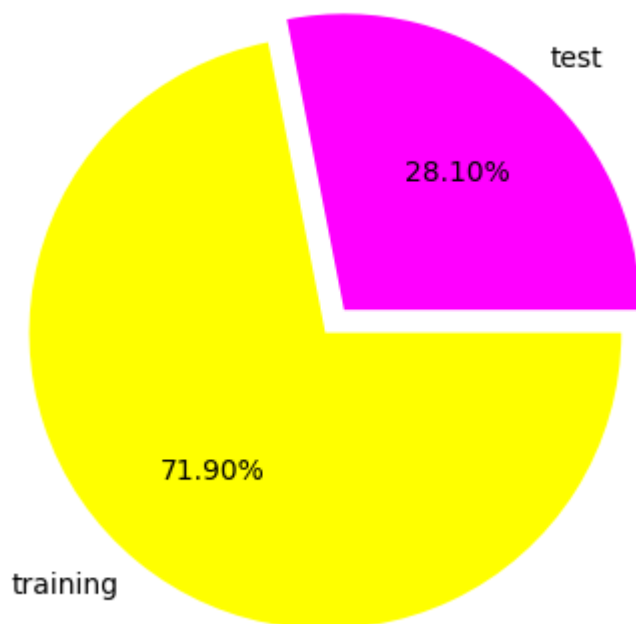
```
In [ ]: df_data = df2.groupby(by = 'data')['trip_uuid'].count().to_frame().reset_index()
df_data['perc'] = np.round(df_data['trip_uuid'] * 100 / df_data['trip_uuid'].sum(), 2)
df_data
```

```
Out[ ]:
```

	data	trip_uuid	perc
0	test	4163	28.1
1	training	10654	71.9

```
In [ ]: plt.pie(x = df_data['trip_uuid'],
labels = df_data['data'],
explode = [0, 0.1],
autopct = '%.2f%%',
colors = ['magenta', 'yellow'])
plt.plot()
```

```
Out[ ]: []
```



Distribution of number of trips created from different states :

```
In [ ]: df_source_state = df2.groupby(by = 'source_state')['trip_uuid'].count().to_frame().reset_index()
df_source_state['perc'] = np.round(df_source_state['trip_uuid'] * 100 / df_source_state['trip_uuid'].sum(), 2)
df_source_state = df_source_state.sort_values(by = 'trip_uuid', ascending = False)
df_source_state
```

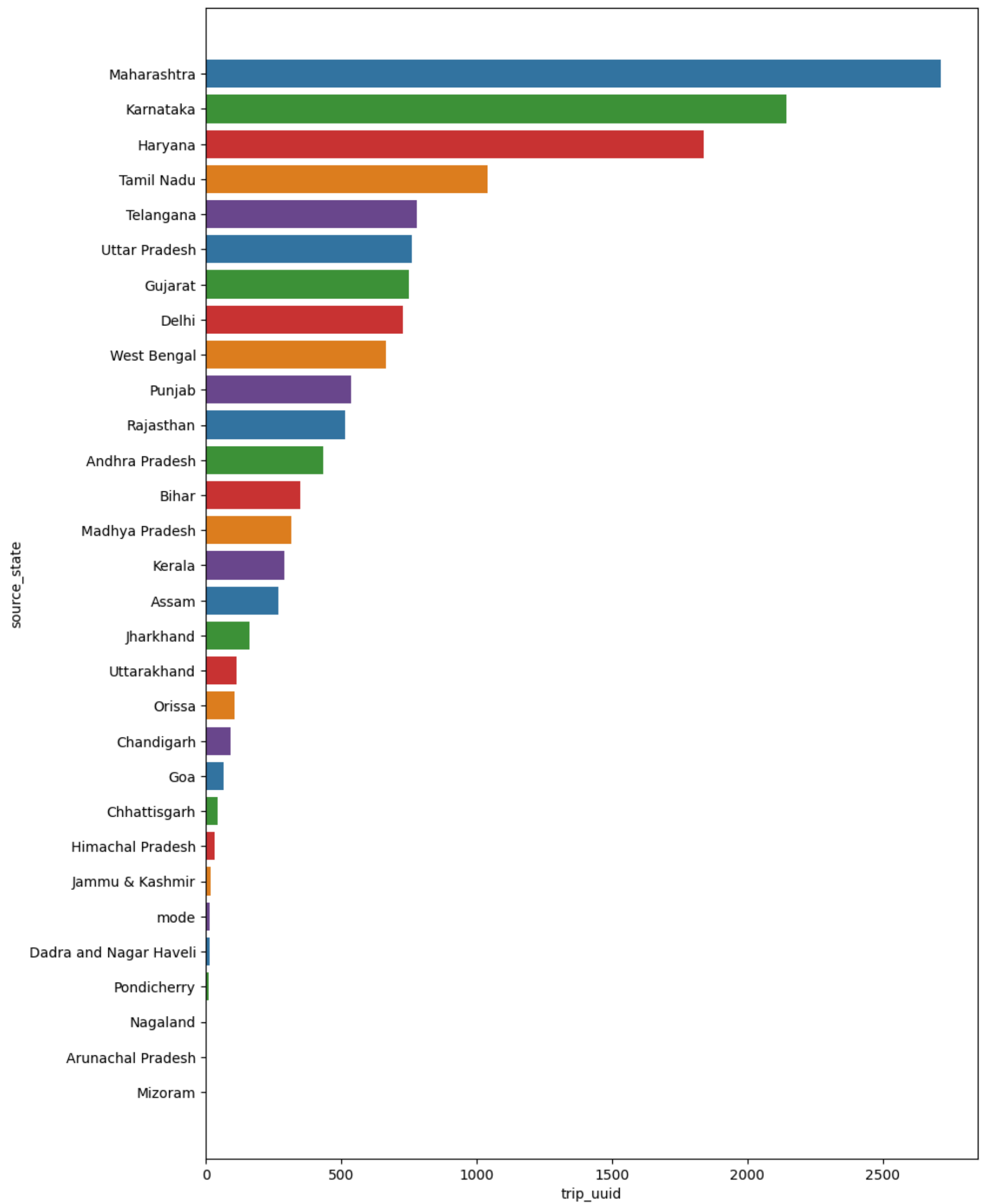

Out[]:

	source_state	trip_uuid	perc
17	Maharashtra	2714	18.32
14	Karnataka	2143	14.46
10	Haryana	1838	12.40
24	Tamil Nadu	1039	7.01
25	Telangana	781	5.27
26	Uttar Pradesh	762	5.14
9	Gujarat	750	5.06
7	Delhi	728	4.91
28	West Bengal	665	4.49
22	Punjab	536	3.62
23	Rajasthan	514	3.47
0	Andhra Pradesh	434	2.93
3	Bihar	350	2.36
16	Madhya Pradesh	317	2.14
15	Kerala	289	1.95
2	Assam	268	1.81
13	Jharkhand	160	1.08
27	Uttarakhand	114	0.77
20	Orissa	107	0.72
4	Chandigarh	93	0.63
8	Goa	65	0.44
5	Chhattisgarh	43	0.29
11	Himachal Pradesh	34	0.23
12	Jammu & Kashmir	17	0.11
29	mode	16	0.11
6	Dadra and Nagar Haveli	15	0.10
21	Pondicherry	12	0.08
19	Nagaland	5	0.03
1	Arunachal Pradesh	4	0.03
18	Mizoram	4	0.03

In []:

```
plt.figure(figsize = (10, 15))
custom_palette = ["#1f78b4", "#33a02c", "#e31a1c", "#ff7f00", "#6a3d9a"]
sns.barplot(data = df_source_state,
            x = df_source_state['trip_uuid'],
            y = df_source_state['source_state'],
            palette=custom_palette)
plt.plot()
```

Out[]: []



Top 10 cities based on the number of trips created from different cities :

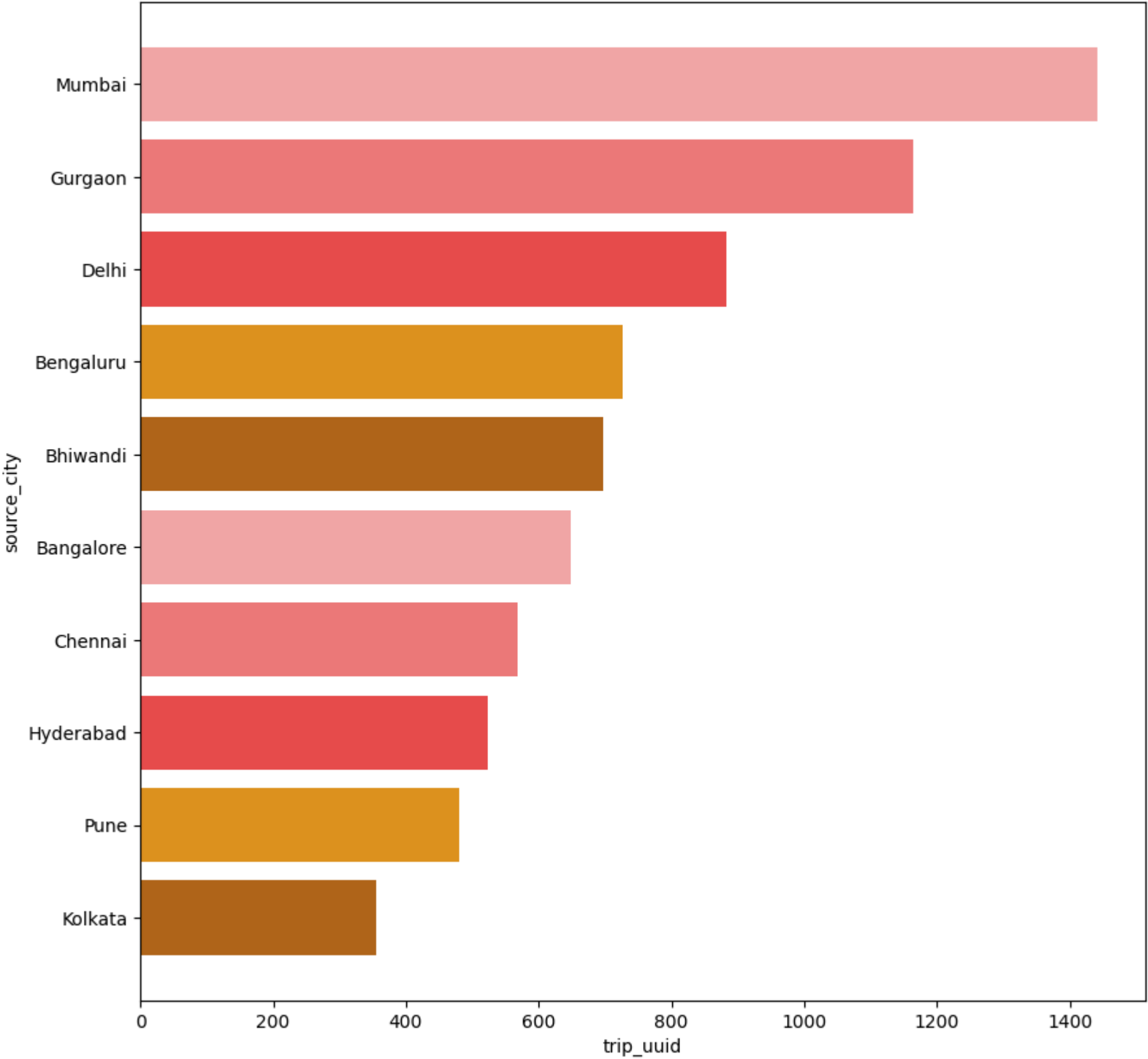
```
In [ ]: df_source_city = df2.groupby(by = 'source_city')['trip_uuid'].count().to_frame().reset_index()
df_source_city['perc'] = np.round(df_source_city['trip_uuid'] * 100 / df_source_city['trip_uuid'].sum(), 2)
df_source_city = df_source_city.sort_values(by = 'trip_uuid', ascending = False)[:10]
df_source_city
```

Out[]:

	source_city	trip_uuid	perc
439	Mumbai	1442	9.73
237	Gurgaon	1165	7.86
169	Delhi	883	5.96
79	Bengaluru	726	4.90
100	Bhiwandi	697	4.70
58	Bangalore	648	4.37
136	Chennai	568	3.83
264	Hyderabad	524	3.54
516	Pune	480	3.24
357	Kolkata	356	2.40

```
In [ ]: plt.figure(figsize = (10, 10))
custom_palette_warm = ["#ff9999", "#ff6666", "#ff3333", "#ff9900", "#cc6600"]
sns.barplot(data = df_source_city,
            x = df_source_city['trip_uuid'],
            y = df_source_city['source_city'],
            palette=custom_palette_warm)
plt.plot()
```

Out[]: []



- It can be seen in the above plot that maximum trips originated from Mumbai city followed by Gurgaon Delhi, Bengaluru and Bhiwandi.

Top 10 states who placed orders :

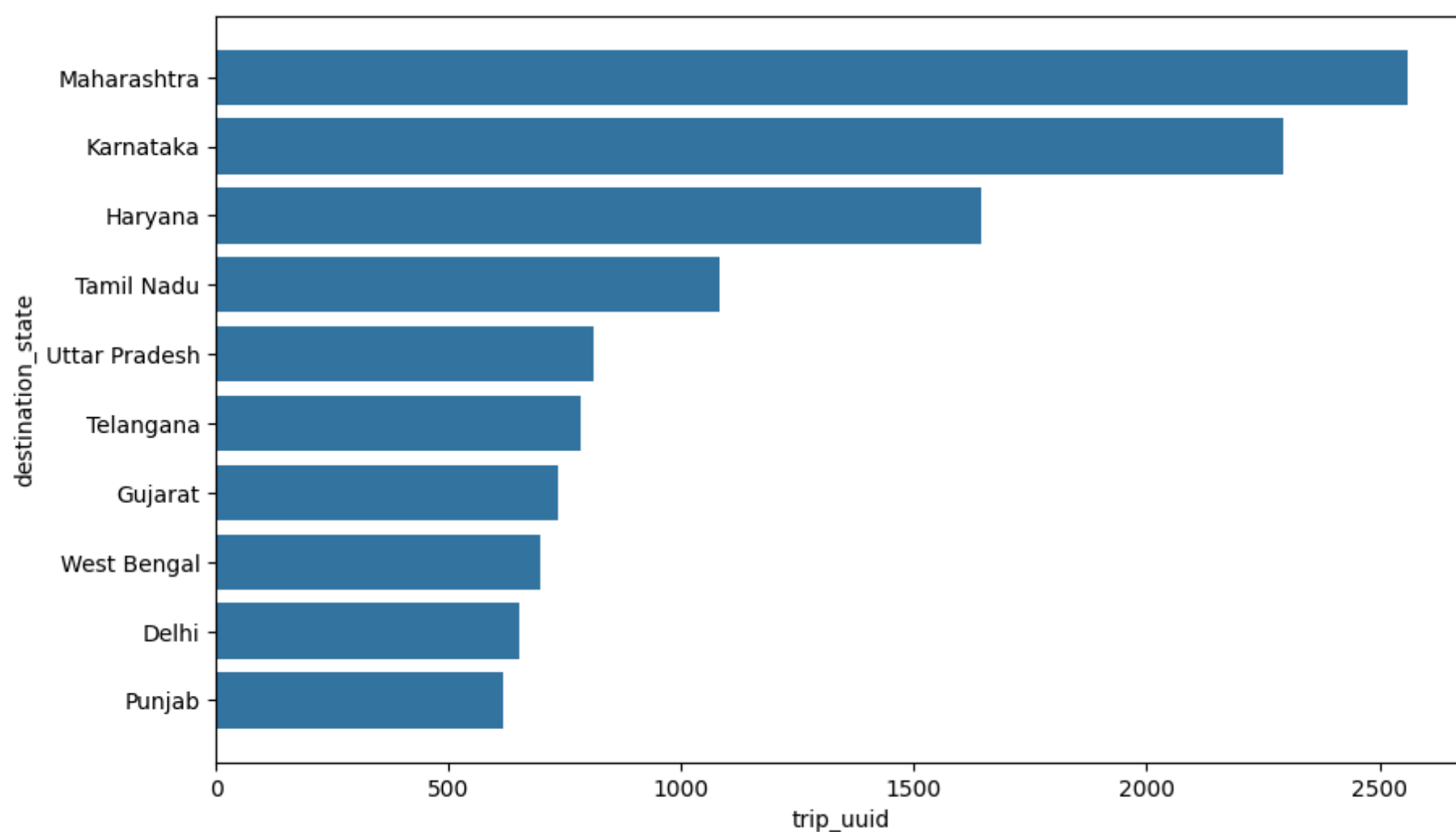
```
In [ ]: df_destination_state = df2.groupby(by = 'destination_state')['trip_uuid'].count().to_frame().reset_index()
df_destination_state['perc'] = np.round(df_destination_state['trip_uuid'] * 100 / df_destination_state['trip_uuid'].sum(), 2)
df_destination_state = df_destination_state.sort_values(by = 'trip_uuid', ascending = False)[:10]
df_destination_state
```

```
Out[ ]:
```

	destination_state	trip_uuid	perc
18	Maharashtra	2561	17.28
15	Karnataka	2294	15.48
11	Haryana	1643	11.09
25	Tamil Nadu	1084	7.32
28	Uttar Pradesh	811	5.47
26	Telangana	784	5.29
10	Gujarat	734	4.95
30	West Bengal	697	4.70
8	Delhi	652	4.40
23	Punjab	617	4.16

```
In [ ]: plt.figure(figsize = (10, 6))
sns.barplot(data = df_destination_state,
            x = df_destination_state['trip_uuid'],
            y = df_destination_state['destination_state'])
plt.plot()
```

```
Out[ ]: []
```



- Maximum trips ended in Maharashtra state followed by Karnataka, Haryana, Tamil Nadu.

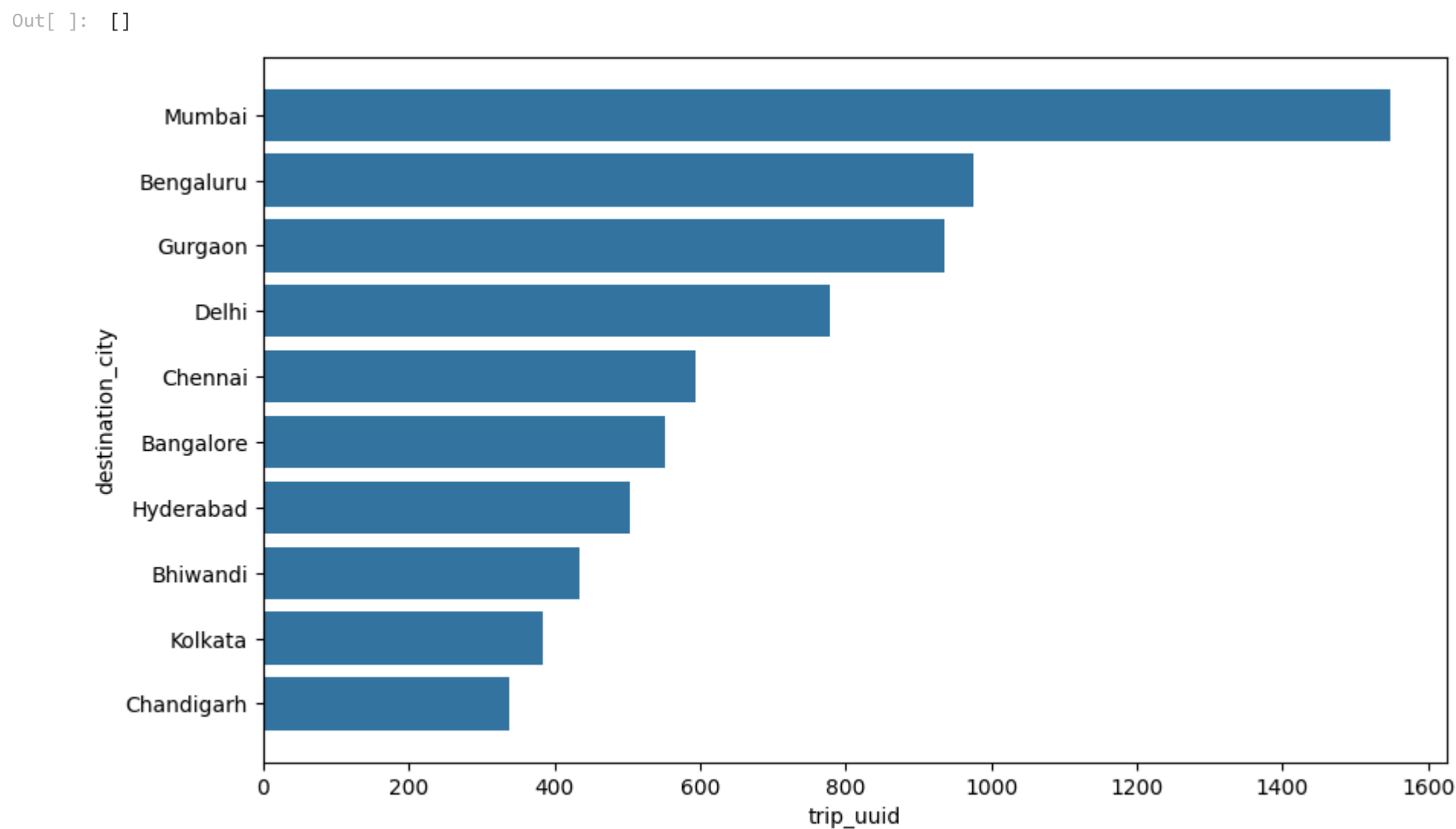
Top 10 cities based on the number of order placed :

```
In [ ]: df_destination_city = df2.groupby(by = 'destination_city')['trip_uuid'].count().to_frame().reset_index()
df_destination_city['perc'] = np.round(df_destination_city['trip_uuid'] * 100 / df_destination_city['trip_uuid'].sum(), 2)
df_destination_city = df_destination_city.sort_values(by = 'trip_uuid', ascending = False)[:10]
df_destination_city
```

```
Out[ ]:
```

	destination_city	trip_uuid	perc
515	Mumbai	1548	10.45
96	Bengaluru	975	6.58
282	Gurgaon	936	6.32
200	Delhi	778	5.25
163	Chennai	595	4.02
72	Bangalore	551	3.72
308	Hyderabad	503	3.39
115	Bhiwandi	434	2.93
418	Kolkata	384	2.59
158	Chandigarh	339	2.29

```
In [ ]: plt.figure(figsize = (10, 6))
sns.barplot(data = df_destination_city,
            x = df_destination_city['trip_uuid'],
            y = df_destination_city['destination_city'])
plt.plot()
```



- It can be seen in the above plot that maximum trips ended in Mumbai city followed by Bengaluru, Gurgaon, Delhi and Chennai. That means that the number of orders placed in these cities is significantly high.

Find outliers in the numerical variables (you might find outliers in almost all the variables), and check it using visual analysis:

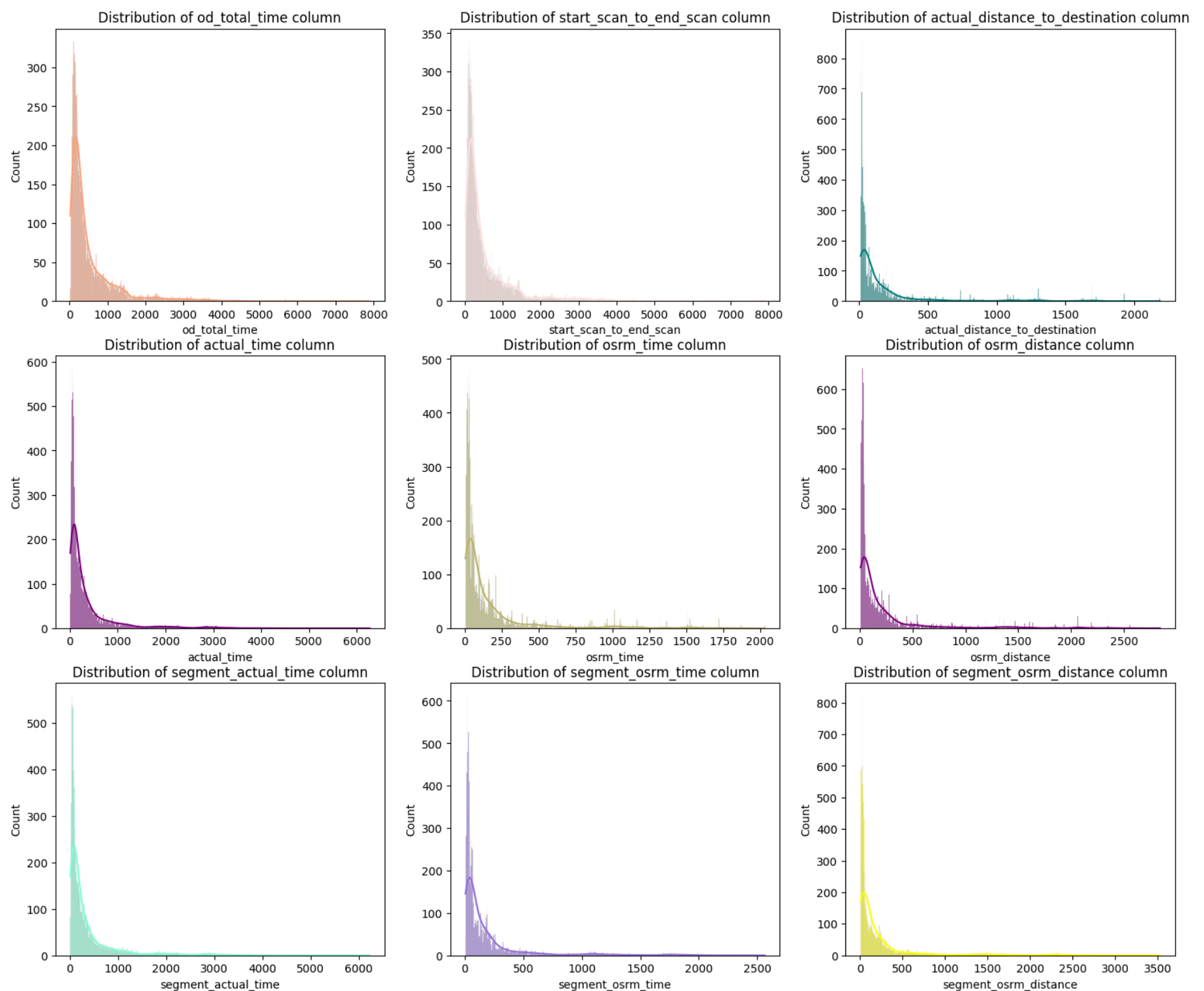
```
In [ ]: numerical_columns = ['od_total_time', 'start_scan_to_end_scan', 'actual_distance_to_destination',
                           'actual_time', 'osrm_time', 'osrm_distance', 'segment_actual_time',
                           'segment_osrm_time', 'segment_osrm_distance']
df2[numerical_columns].describe().T
```

	count	mean	std	min	25%	50%	75%	max
od_total_time	14817.0	531.697630	658.868223	23.460000	149.930000	280.770000	638.200000	7898.550000
start_scan_to_end_scan	14817.0	530.810016	658.705957	23.000000	149.000000	280.000000	637.000000	7898.000000
actual_distance_to_destination	14817.0	164.477838	305.388147	9.002461	22.837239	48.474072	164.583208	2186.531787
actual_time	14817.0	357.143754	561.396157	9.000000	67.000000	149.000000	370.000000	6265.000000
osrm_time	14817.0	161.384018	271.360995	6.000000	29.000000	60.000000	168.000000	2032.000000
osrm_distance	14817.0	204.344689	370.395573	9.072900	30.819200	65.618800	208.475000	2840.081000
segment_actual_time	14817.0	353.892286	556.247965	9.000000	66.000000	147.000000	367.000000	6230.000000
segment_osrm_time	14817.0	180.949787	314.542047	6.000000	31.000000	65.000000	185.000000	2564.000000
segment_osrm_distance	14817.0	223.201161	416.628374	9.072900	32.654500	70.154400	218.802400	3523.632400

```

In [ ]: plt.figure(figsize = (18, 15))
for i in range(len(numerical_columns)):
    plt.subplot(3, 3, i + 1)
    clr = np.random.choice(list(mpl.colors.cnames))
    sns.histplot(df2[numerical_columns[i]], bins = 1000, kde = True, color = clr)
    plt.title(f"Distribution of {numerical_columns[i]} column")
    plt.plot()

```

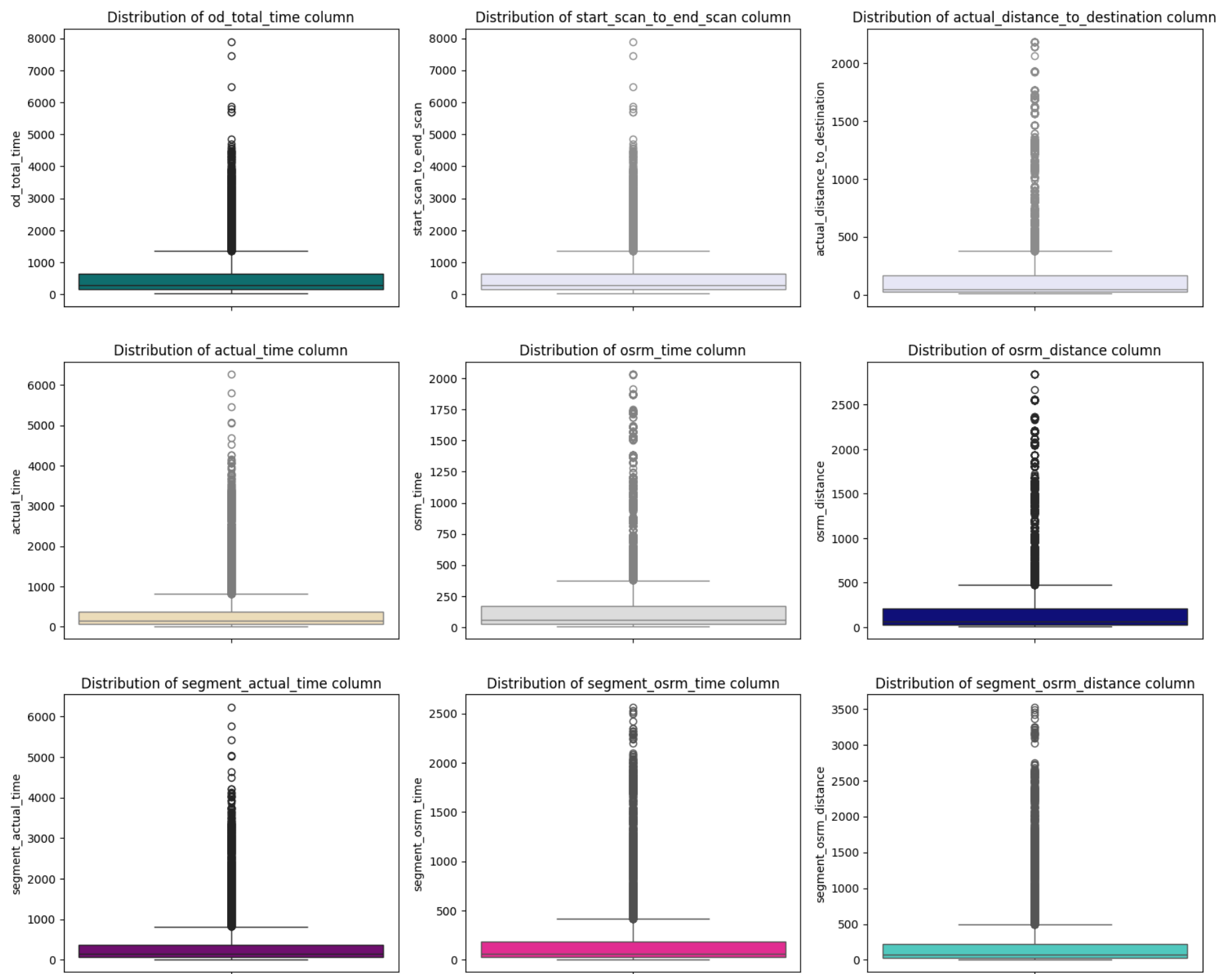


-It can be inferred from the above plots that data in all the numerical columns are right skewed.

```

In [ ]: plt.figure(figsize = (18, 15))
for i in range(len(numerical_columns)):
    plt.subplot(3, 3, i + 1)
    clr = np.random.choice(list(mpl.colors.cnames))
    sns.boxplot(df2[numerical_columns[i]], color = clr)
    plt.title(f"Distribution of {numerical_columns[i]} column")
    plt.plot()

```

-It can be clearly seen in the above plots that there are outliers in all the numerical columns that need to be treated.

```
In [ ]: # Detecting Outliers

for i in numerical_columns:
    Q1 = np.quantile(df2[i], 0.25)
    Q3 = np.quantile(df2[i], 0.75)
    IQR = Q3 - Q1
    LB = Q1 - 1.5 * IQR
    UB = Q3 + 1.5 * IQR
    outliers = df2.loc[(df2[i] < LB) | (df2[i] > UB)]
    print('Column:', i)
    print(f'Q1 : {Q1}')
    print(f'Q3 : {Q3}')
    print(f'IQR : {IQR}')
    print(f'LB : {LB}')
    print(f'UB : {UB}')
    print(f'Number of outliers : {outliers.shape[0]}')
    print('-----')
```

```

Column : od_total_time
Q1 : 149.93
Q3 : 638.2
IQR : 488.27000000000004
LB : -582.4750000000001
UB : 1370.605
Number of outliers : 1266
-----
Column : start_scan_to_end_scan
Q1 : 149.0
Q3 : 637.0
IQR : 488.0
LB : -583.0
UB : 1369.0
Number of outliers : 1267
-----
Column : actual_distance_to_destination
Q1 : 22.83723905859321
Q3 : 164.58320763841138
IQR : 141.74596857981817
LB : -189.78171381113404
UB : 377.2021605081386
Number of outliers : 1449
-----
Column : actual_time
Q1 : 67.0
Q3 : 370.0
IQR : 303.0
LB : -387.5
UB : 824.5
Number of outliers : 1643
-----
Column : osrm_time
Q1 : 29.0
Q3 : 168.0
IQR : 139.0
LB : -179.5
UB : 376.5
Number of outliers : 1517
-----
Column : osrm_distance
Q1 : 30.8192
Q3 : 208.475
IQR : 177.6558
LB : -235.6645
UB : 474.9587
Number of outliers : 1524
-----
Column : segment_actual_time
Q1 : 66.0
Q3 : 367.0
IQR : 301.0
LB : -385.5
UB : 818.5
Number of outliers : 1643
-----
Column : segment_osrm_time
Q1 : 31.0
Q3 : 185.0
IQR : 154.0
LB : -200.0
UB : 416.0
Number of outliers : 1492
-----
Column : segment_osrm_distance
Q1 : 32.6545
Q3 : 218.8024
IQR : 186.1479
LB : -246.56735000000003
UB : 498.02425000000005
Number of outliers : 1548
-----

```

- The outliers present in our sample data can be the true outliers. It's best to remove outliers only when there is a sound reason for doing so. Some outliers represent natural variations in the population, and they should be left as is in the dataset.

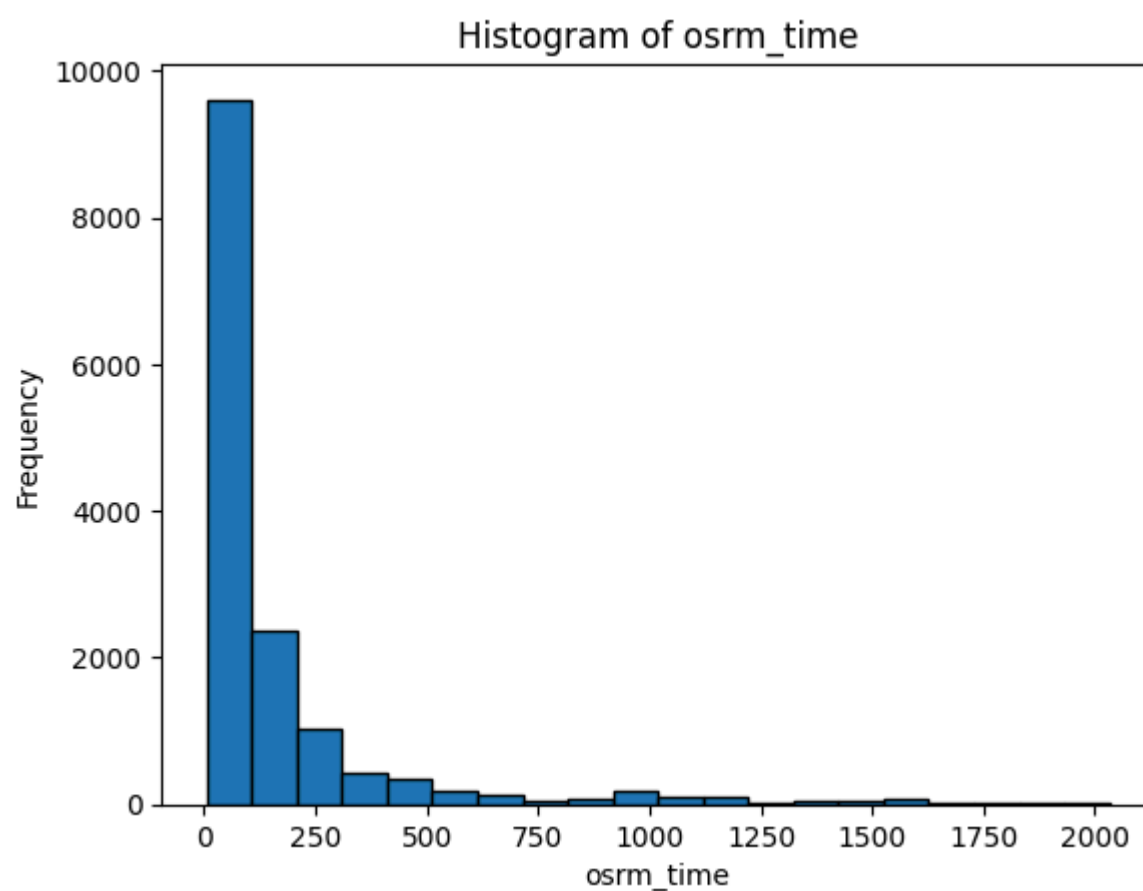
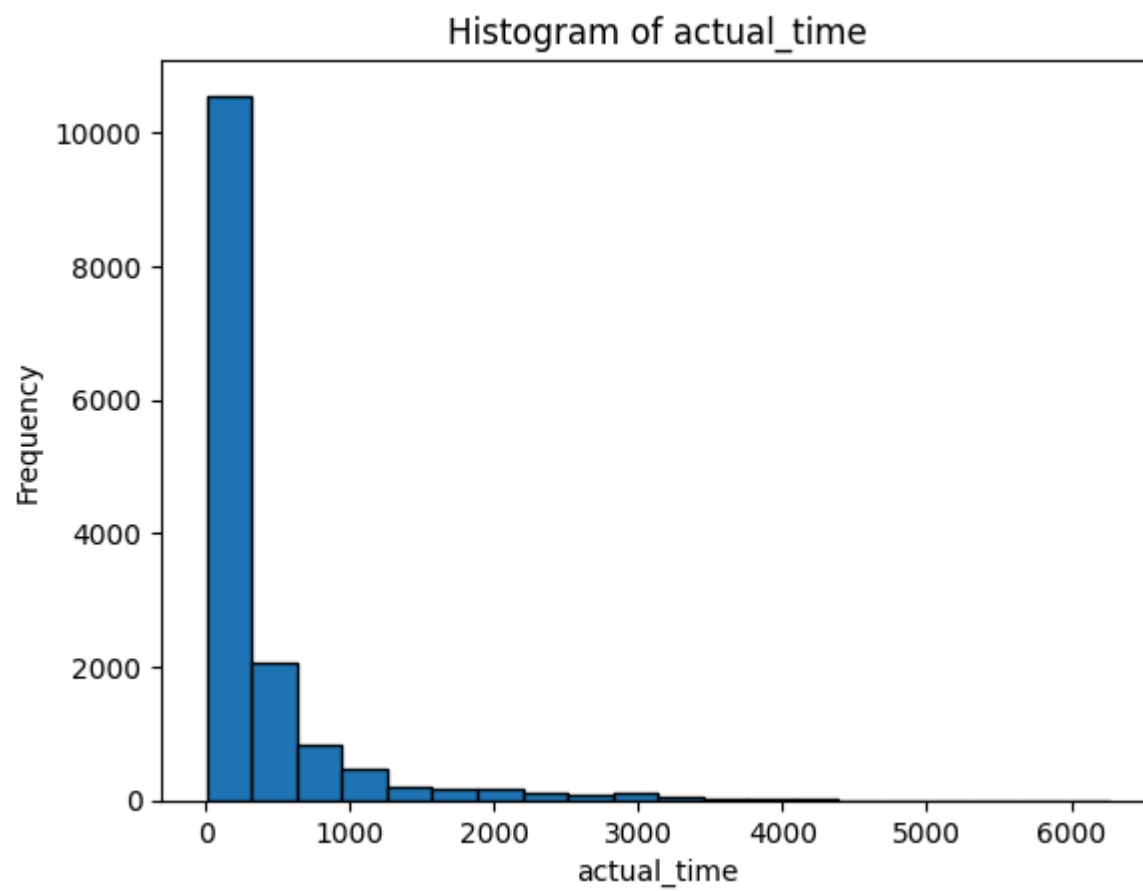
Hypothesis testing :

```
In [ ]: from scipy.stats import ttest_ind
```

```
In [ ]: for column_name in ["actual_time", "osrm_time"]:
# Create a histogram for each column using plt.histplot()
plt.hist(df2[column_name], bins=20, edgecolor='black')
plt.xlabel(column_name) # Set the x-axis label
plt.ylabel("Frequency") # Set the y-axis label
```



```
plt.title(f"Histogram of {column_name}") # Set the plot title
plt.show()
```



```
In [ ]: # Perform the two-sample t-test
#H0: No difference between actual_time and osrm_time
#sig level=0.05
ttest_ind(df2['actual_time'], df2['osrm_time'])
```

```
Out[ ]: TtestResult(statistic=np.float64(38.215453905833165), pvalue=np.float64(0.0), df=np.float64(29632.0))
```

-The p-value is less than the significance level of 0.05, which means we reject the null hypothesis. We can conclude that there is significant difference in the mean actual_time and osrm_time.

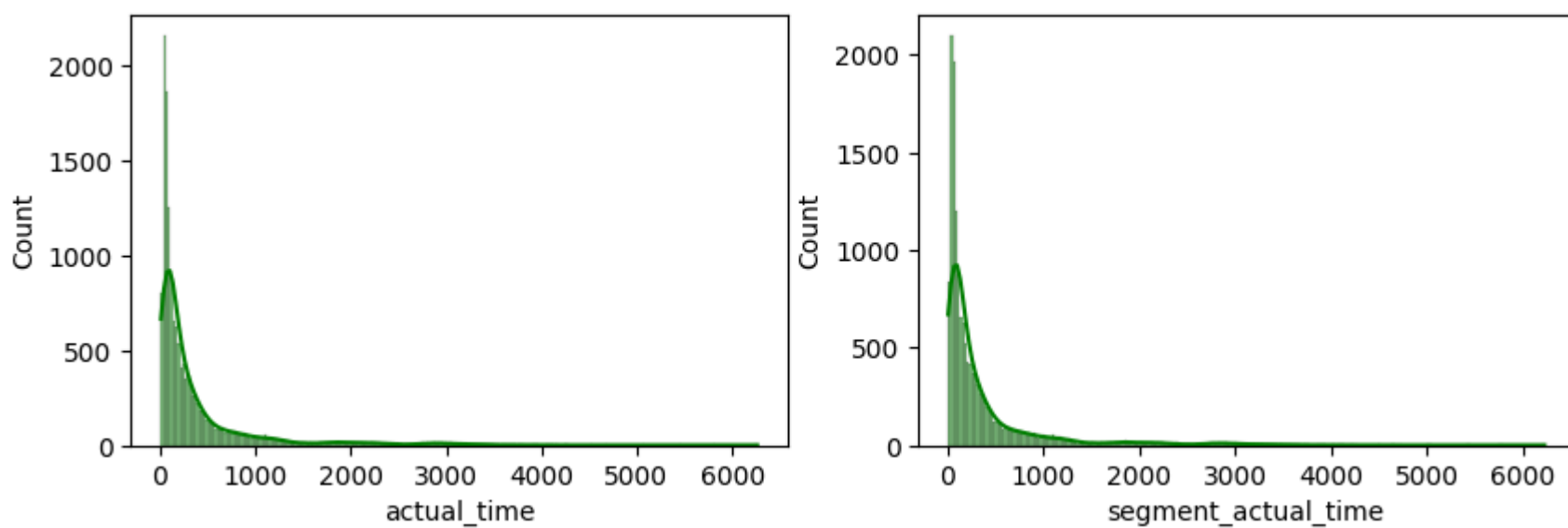
```
In [ ]: #H0: mean(actual_time) > mean(osrm_time)
ttest_ind(df2['actual_time'], df2['osrm_time'], alternative="greater")
```

```
Out[ ]: TtestResult(statistic=np.float64(38.215453905833165), pvalue=np.float64(0.0), df=np.float64(29632.0))
```

-The p-value is less than the significance level of 0.05, which means we reject the null hypothesis. Mean(actual_time) < Mean(osrm_time)
Estimated time between two points is greater than actual time.

Actual_time and Segment_actual_time :

```
In [ ]: plt.figure(figsize=(10,3))
plt.subplot(121)
sns.histplot(df2['actual_time'], kde=True, color='green')
plt.subplot(122)
sns.histplot(df2['segment_actual_time'], kde=True, color='green')
plt.show()
```



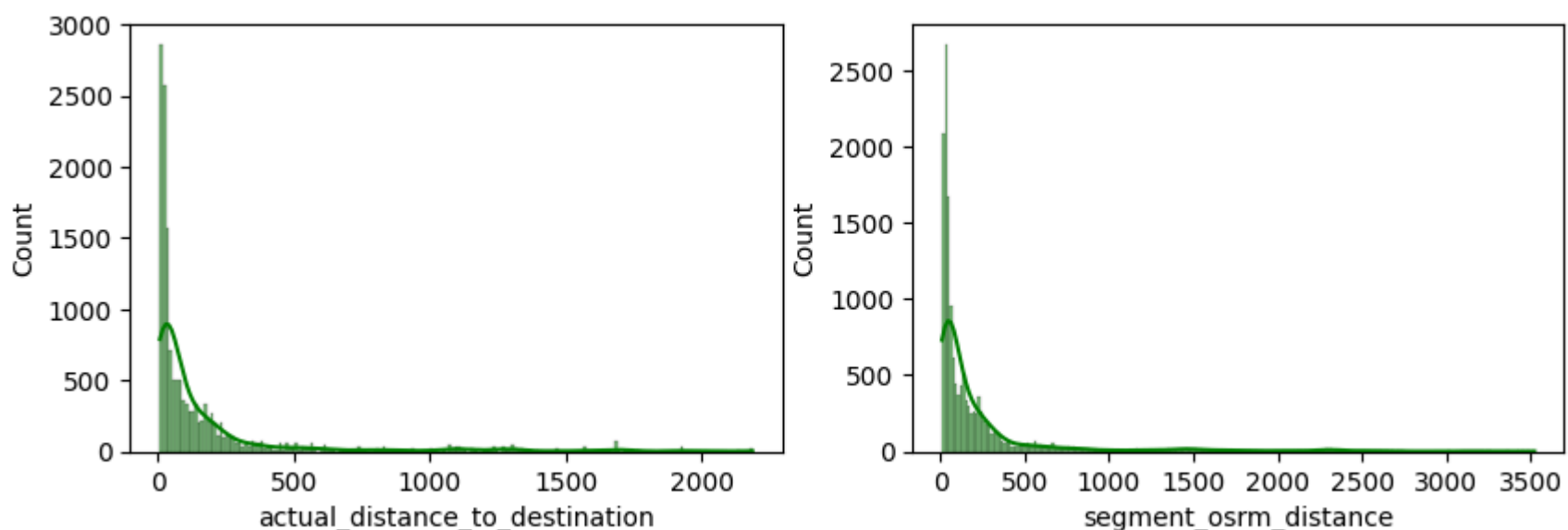
```
In [ ]: # Perform the two-sample t-test
#H0: No difference between actual_time and segment_actual_time
ttest_ind(df2['actual_time'], df2['segment_actual_time'])
```

```
Out[ ]: TtestResult(statistic=np.float64(0.5008024728897531), pvalue=np.float64(0.6165138648224772), df=np.float64(29632.0))
```

-The p-value is greater than the significance level of 0.05, which means we fail to reject the null hypothesis. We can conclude that there is no significant difference in the mean actual_time and cumulative segment_actual_time.

Actual_distance_to_destination and Segment_osrm_distance :

```
In [ ]: plt.figure(figsize=(10,3))
plt.subplot(121)
sns.histplot(df2['actual_distance_to_destination'], kde=True, color='green')
plt.subplot(122)
sns.histplot(df2['segment_osrm_distance'], kde=True, color='green')
plt.show()
```



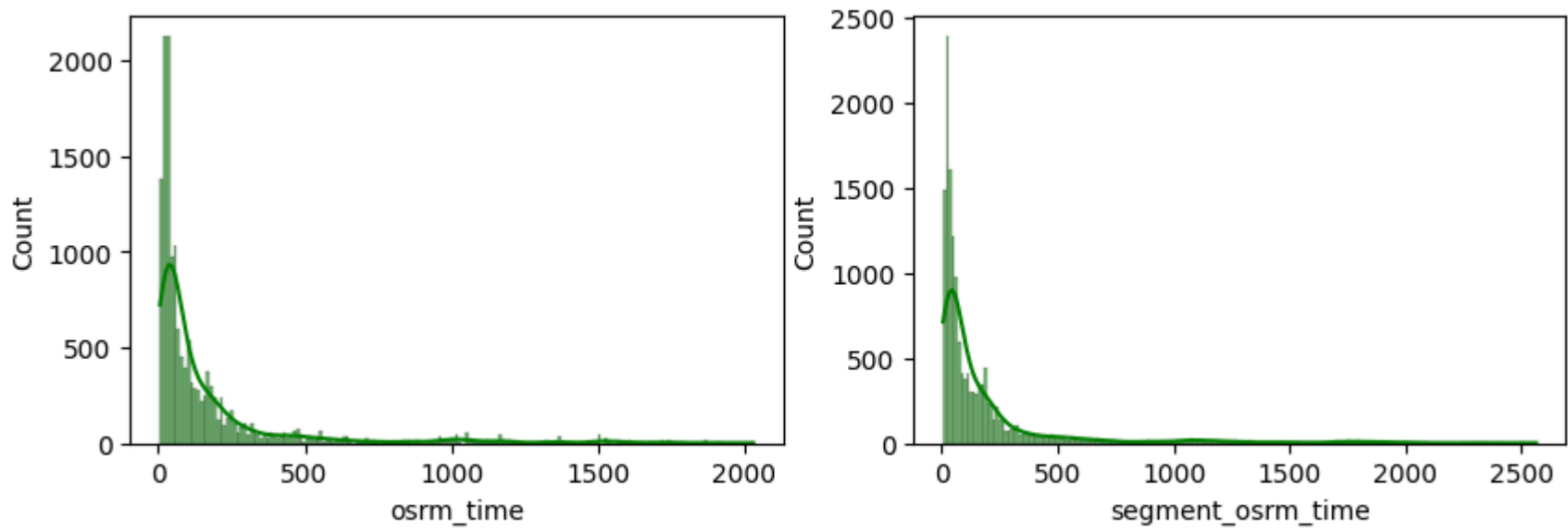
```
In [ ]: # Perform the two-sample t-test
#H0: No difference between cumulative actual_distance_to_destination and cumulative segment_osrm_distance
ttest_ind(df2['actual_distance_to_destination'], df2['segment_osrm_distance'])
```

```
Out[ ]: TtestResult(statistic=np.float64(-13.837715192612745), pvalue=np.float64(2.0603166018745123e-43), df=np.float64(29632.0))
```

-The p-value is less than the significance level of 0.05, which means we reject the null hypothesis. We can conclude that there is significant difference in the mean of cumulative actual_distance_to_destination and cumulative segment_osrm_distance.

Osrm_time and Segment_osrm_time

```
In [ ]: plt.figure(figsize=(10,3))
plt.subplot(121)
sns.histplot(df2['osrm_time'], kde=True, color='green')
plt.subplot(122)
sns.histplot(df2['segment_osrm_time'], kde=True, color='green')
plt.show()
```



```
In [ ]: # Perform the two-sample t-test
#H0: No difference between osrm_distance and cumulated segment_osrm_distance
ttest_ind(df['osrm_time'], df['segment_osrm_time'])
```

```
Out[ ]: TtestResult(statistic=np.float64(241.13291146910316), pvalue=np.float64(0.0), df=np.float64(289732.0))
```

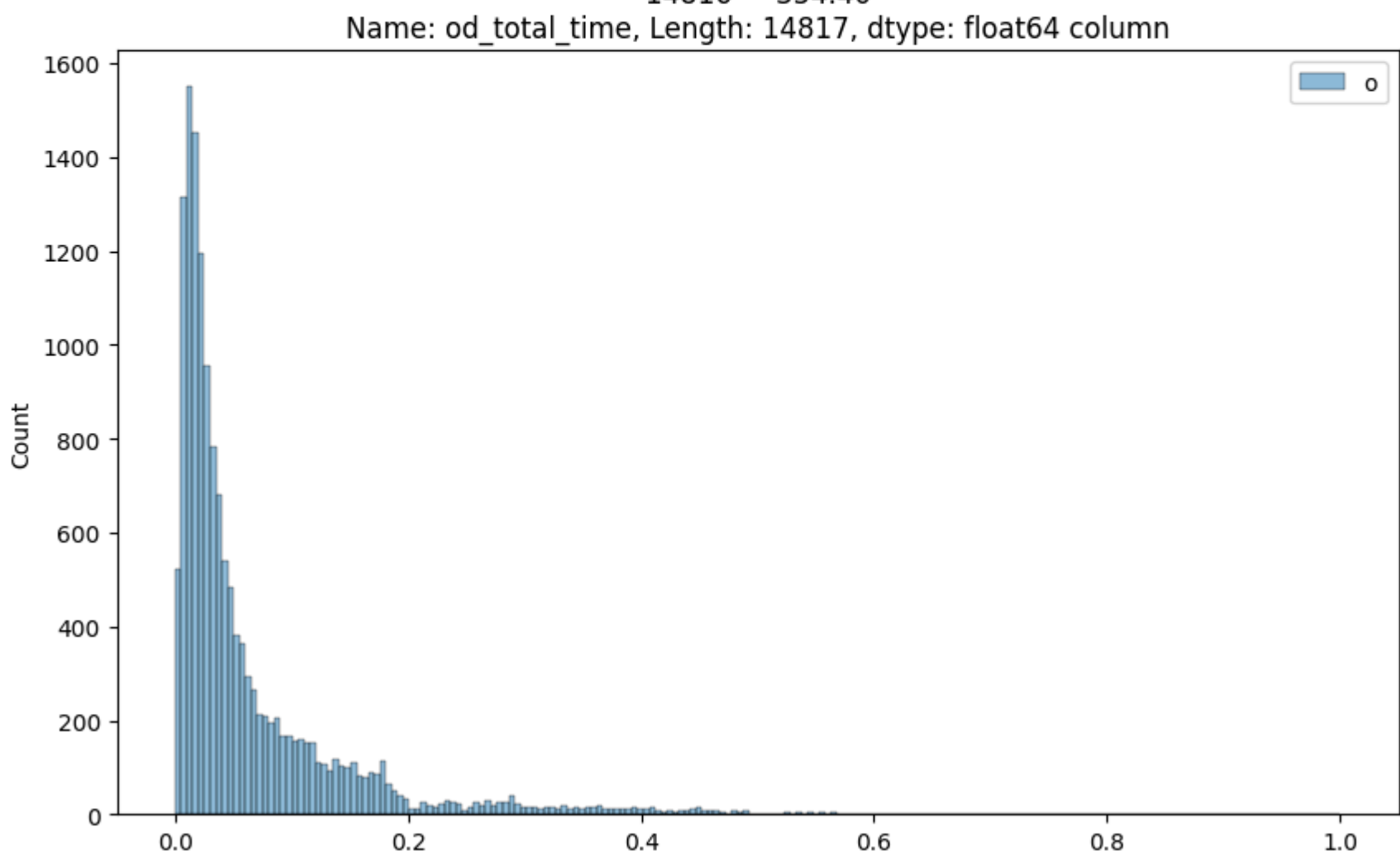
Normalize/ Standardize the numerical features using MinMaxScaler or StandardScaler.

```
In [ ]: from sklearn.preprocessing import MinMaxScaler
```

```
In [ ]: plt.figure(figsize = (10, 6))
scaler = MinMaxScaler()
scaled = scaler.fit_transform(df2['od_total_time'].to_numpy().reshape(-1, 1))
sns.histplot(scaled)
plt.title(f"Normalized {df2['od_total_time']} column")
plt.legend('od_total_time')
plt.plot()
```

```
Out[ ]: []
```

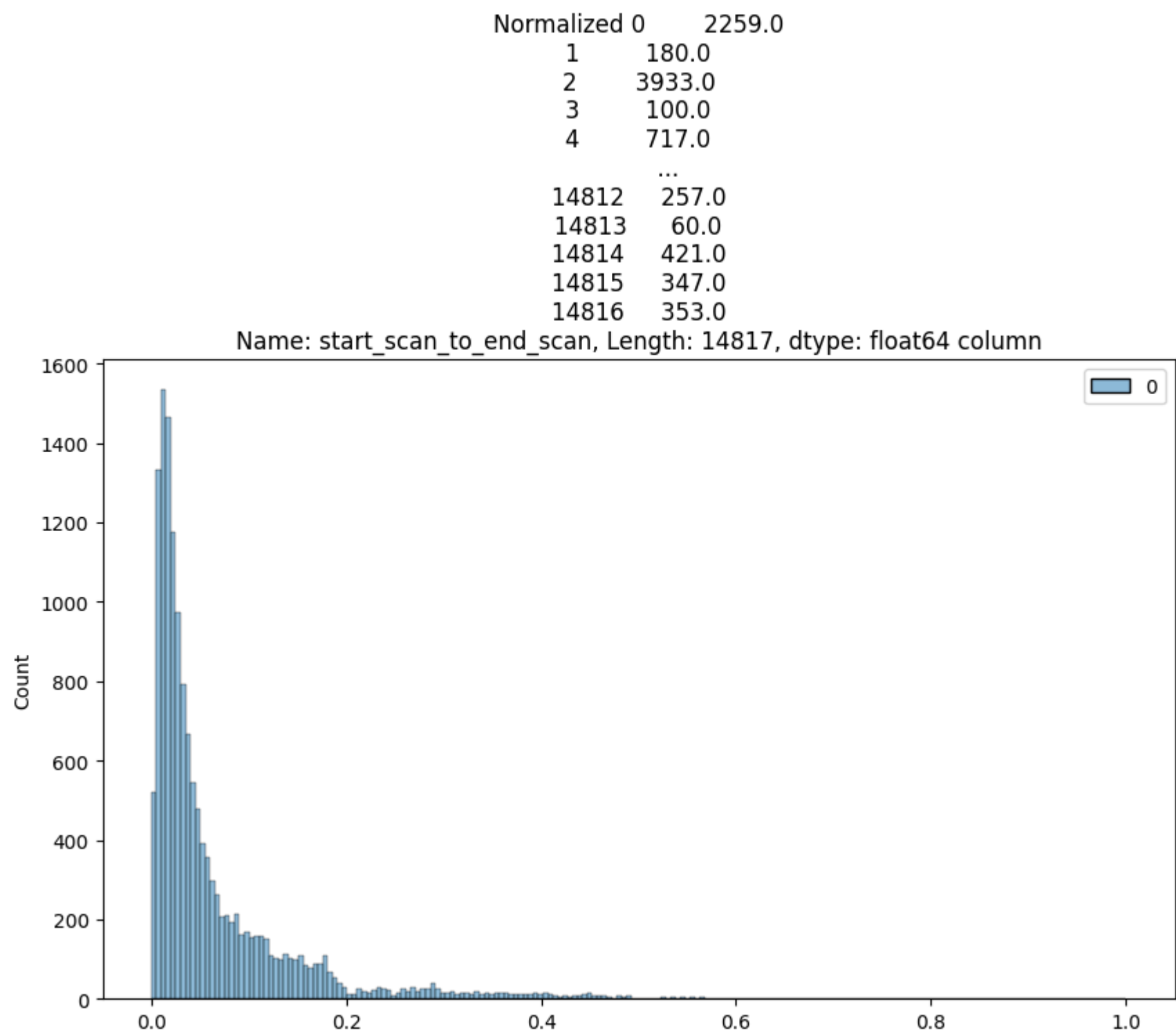
Normalized 0	2260.11
1	181.61
2	3934.36
3	100.49
4	718.34
	...
14812	258.03
14813	60.59
14814	422.12
14815	348.52
14816	354.40



```
In [ ]: plt.figure(figsize = (10, 6))
scaler = MinMaxScaler()
```

```
scaled = scaler.fit_transform(df2['start_scan_to_end_scan'].to_numpy().reshape(-1, 1))
sns.histplot(scaled)
plt.title(f"Normalized {df2['start_scan_to_end_scan']} column")
plt.plot()
```

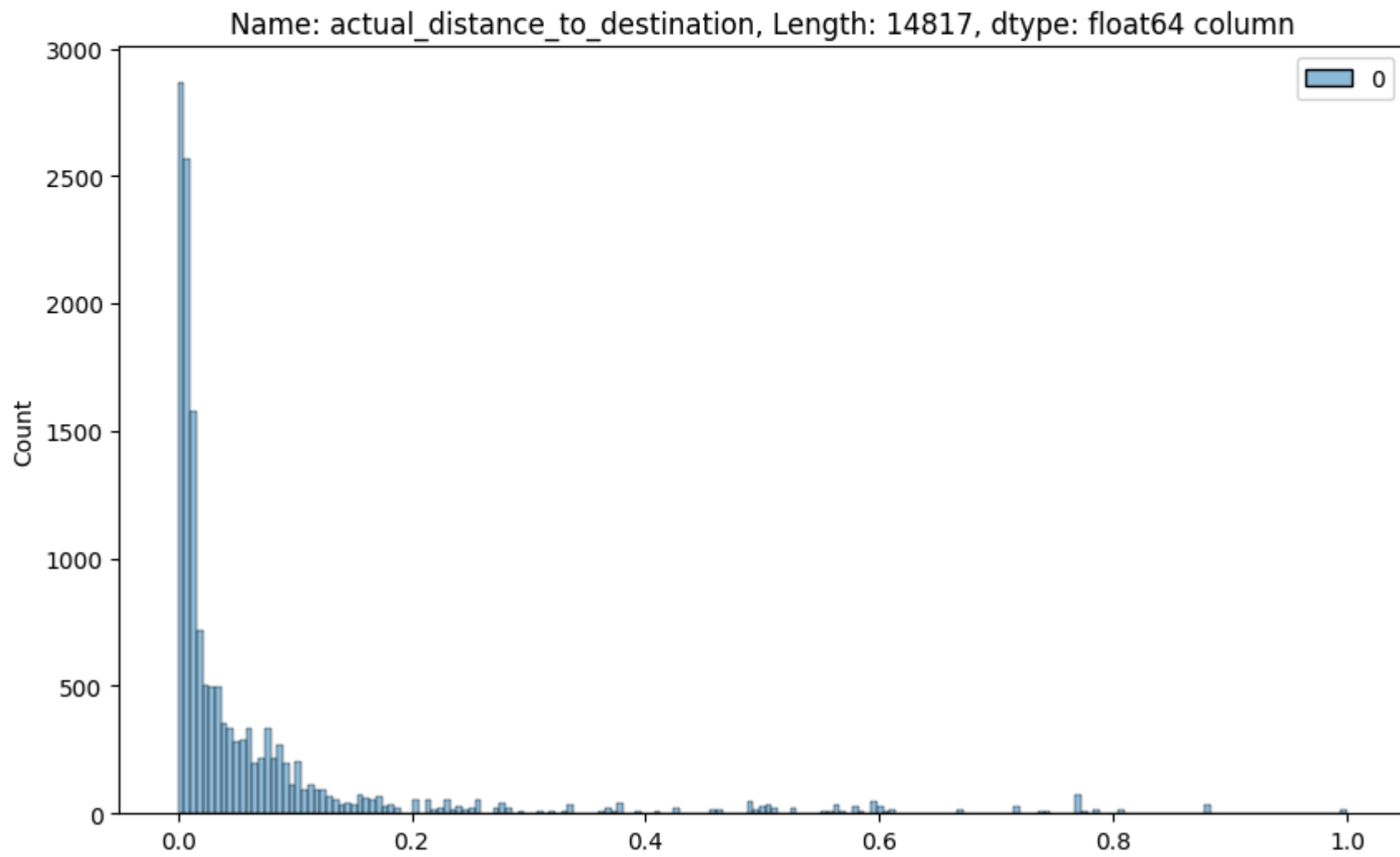
Out[]: []



```
In [ ]: plt.figure(figsize = (10, 6))
scaler = MinMaxScaler()
scaled = scaler.fit_transform(df2['actual_distance_to_destination'].to_numpy().reshape(-1, 1))
sns.histplot(scaled)
plt.title(f"Normalized {df2['actual_distance_to_destination']} column")
plt.plot()
```

Out[]: []

Normalized 0	824.732854
1	73.186911
2	1927.404273
3	17.175274
4	127.448500
	...
14812	57.762332
14813	15.513784
14814	38.684839
14815	134.723836
14816	66.081533

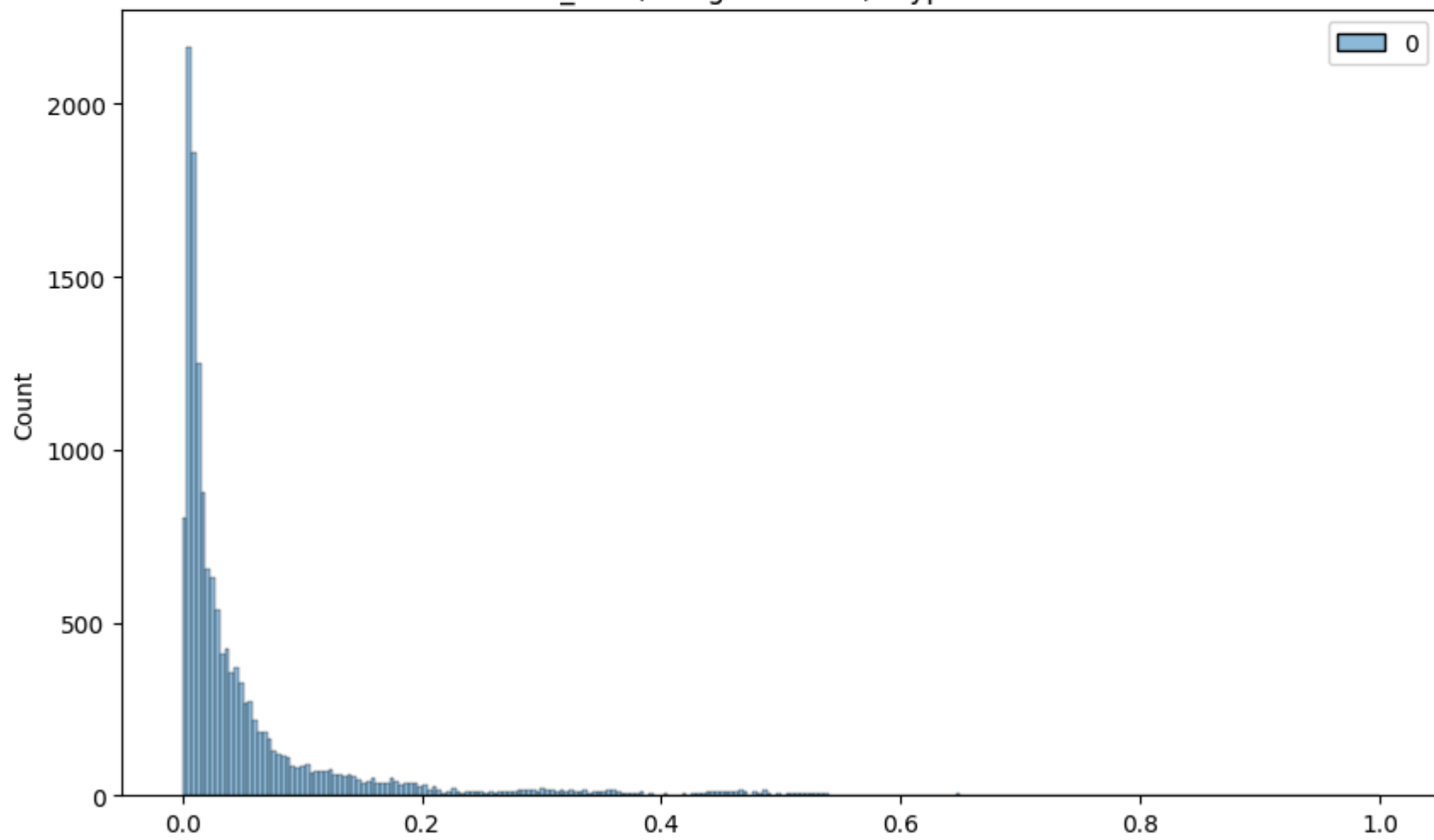


```
In [ ]: plt.figure(figsize = (10, 6))
scaler = MinMaxScaler()
scaled = scaler.fit_transform(df2['actual_time'].to_numpy().reshape(-1, 1))
sns.histplot(scaled)
plt.title(f"Normalized {df2['actual_time']} column")
plt.plot()
```

Out[]: []

Normalized 0	1562.0
1	143.0
2	3347.0
3	59.0
4	341.0
	...
14812	83.0
14813	21.0
14814	282.0
14815	264.0
14816	275.0

Name: actual_time, Length: 14817, dtype: float64 column

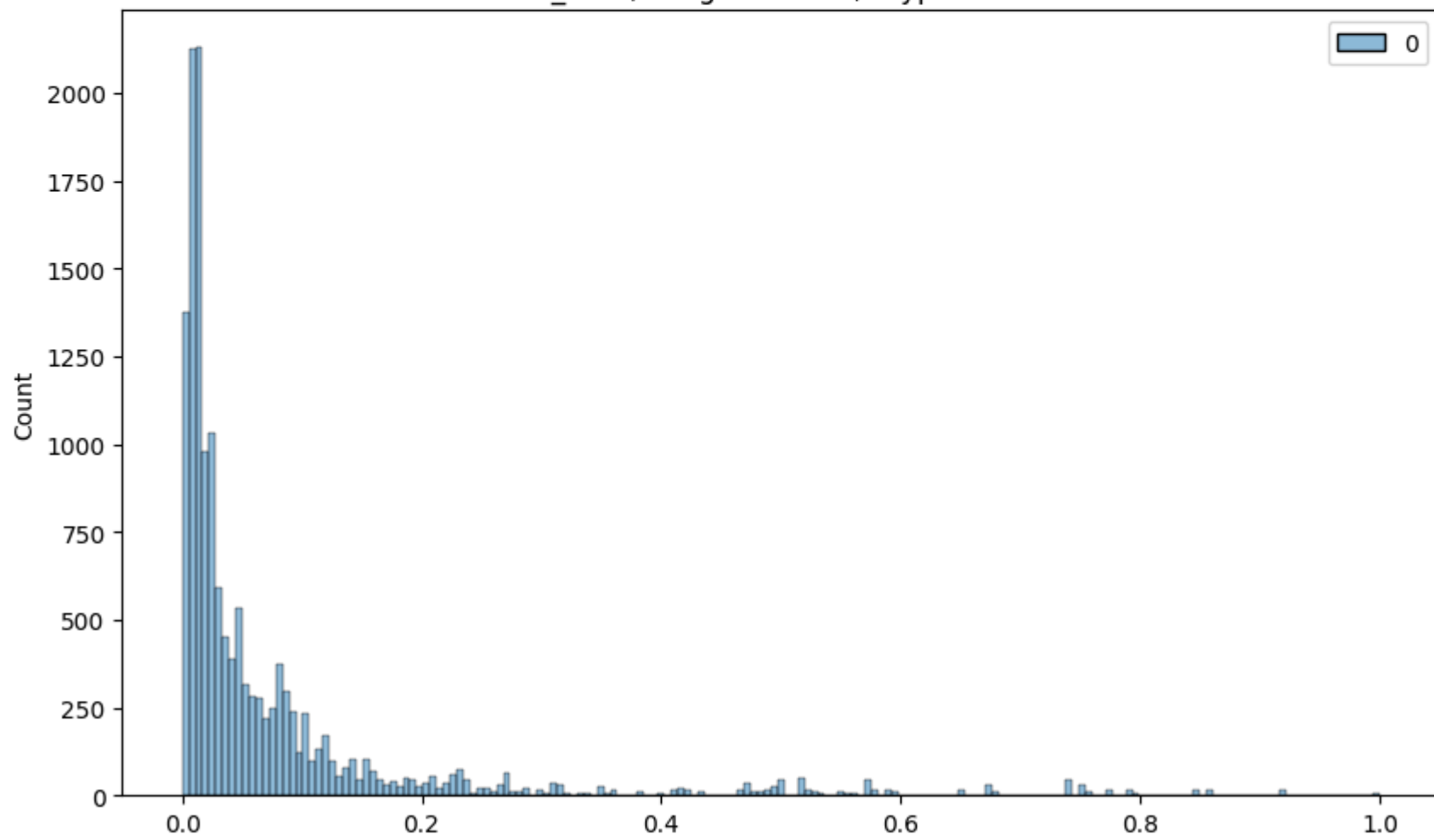


```
In [ ]: plt.figure(figsize = (10, 6))
scaler = MinMaxScaler()
scaled = scaler.fit_transform(df2['osrm_time'].to_numpy().reshape(-1, 1))
sns.histplot(scaled)
plt.title(f"Normalized {df2['osrm_time']} column")
plt.plot()
```

Out[]: []

Normalized	0	717.0
1	68.0	
2	1740.0	
3	15.0	
4	117.0	
	...	
14812	62.0	
14813	12.0	
14814	48.0	
14815	179.0	
14816	68.0	

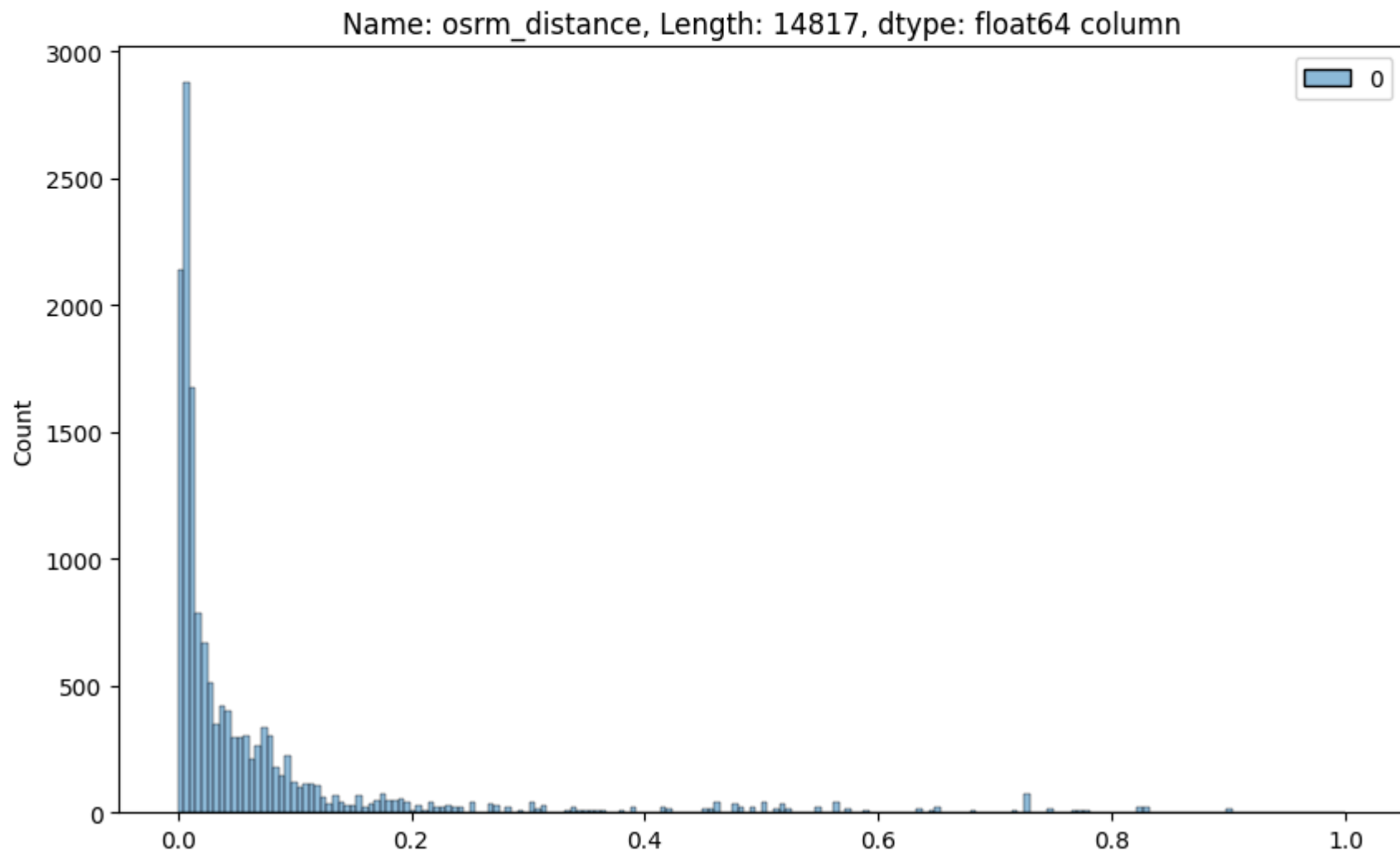
Name: osrm_time, Length: 14817, dtype: float64 column



```
In [ ]: plt.figure(figsize = (10, 6))
scaler = MinMaxScaler()
scaled = scaler.fit_transform(df2['osrm_distance'].to_numpy().reshape(-1, 1))
sns.histplot(scaled)
plt.title(f"Normalized {df2['osrm_distance']} column")
plt.plot()
```

Out[]: []

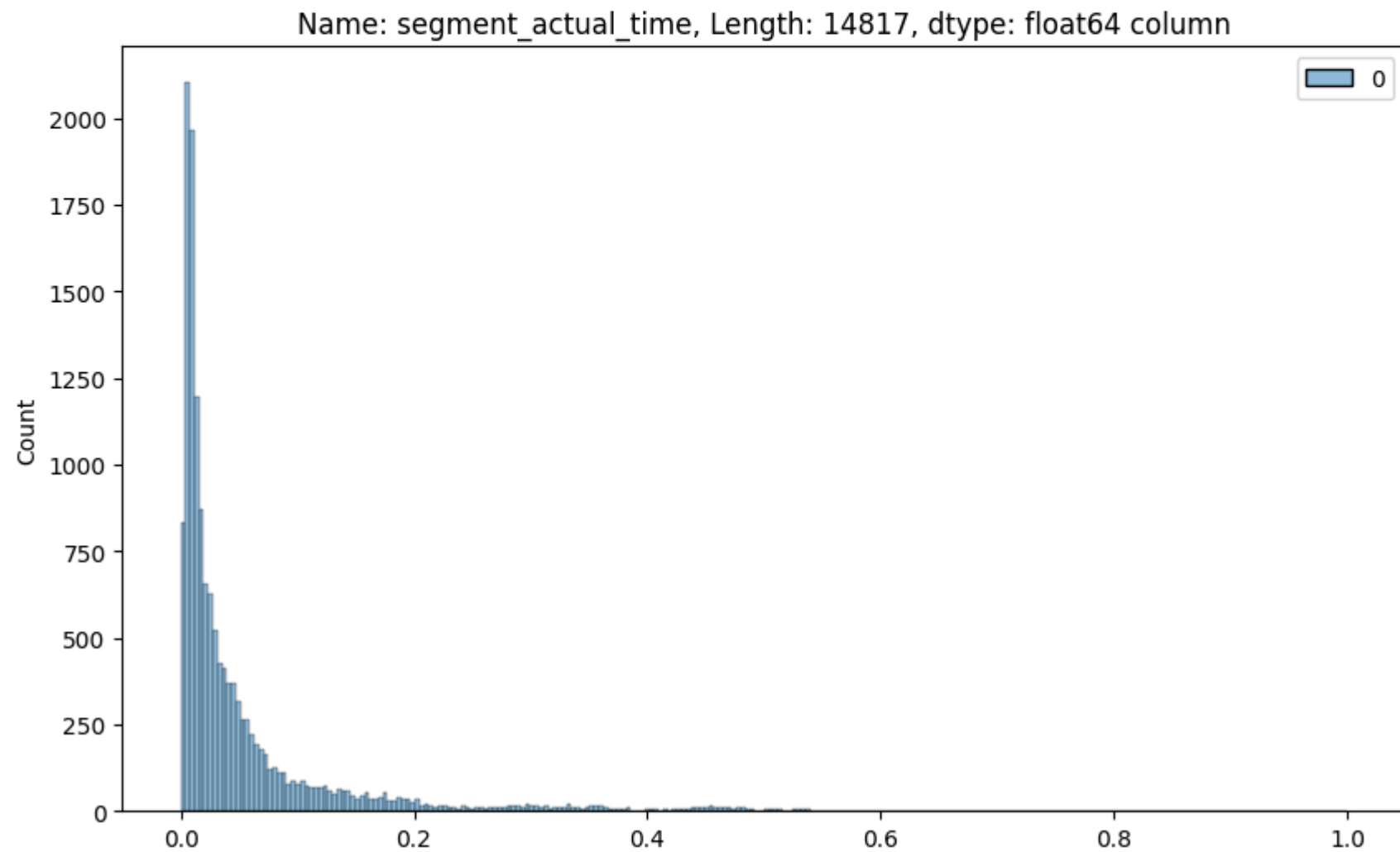
Normalized 0	991.3523
1	85.1110
2	2354.0665
3	19.6800
4	146.7918
	...
14812	73.4630
14813	16.0882
14814	58.9037
14815	171.1103
14816	80.5787



```
In [ ]: plt.figure(figsize = (10, 6))
scaler = MinMaxScaler()
scaled = scaler.fit_transform(df2['segment_actual_time'].to_numpy().reshape(-1, 1))
sns.histplot(scaled)
plt.title(f"Normalized {df2['segment_actual_time']} column")
plt.plot()
```

Out[]: []

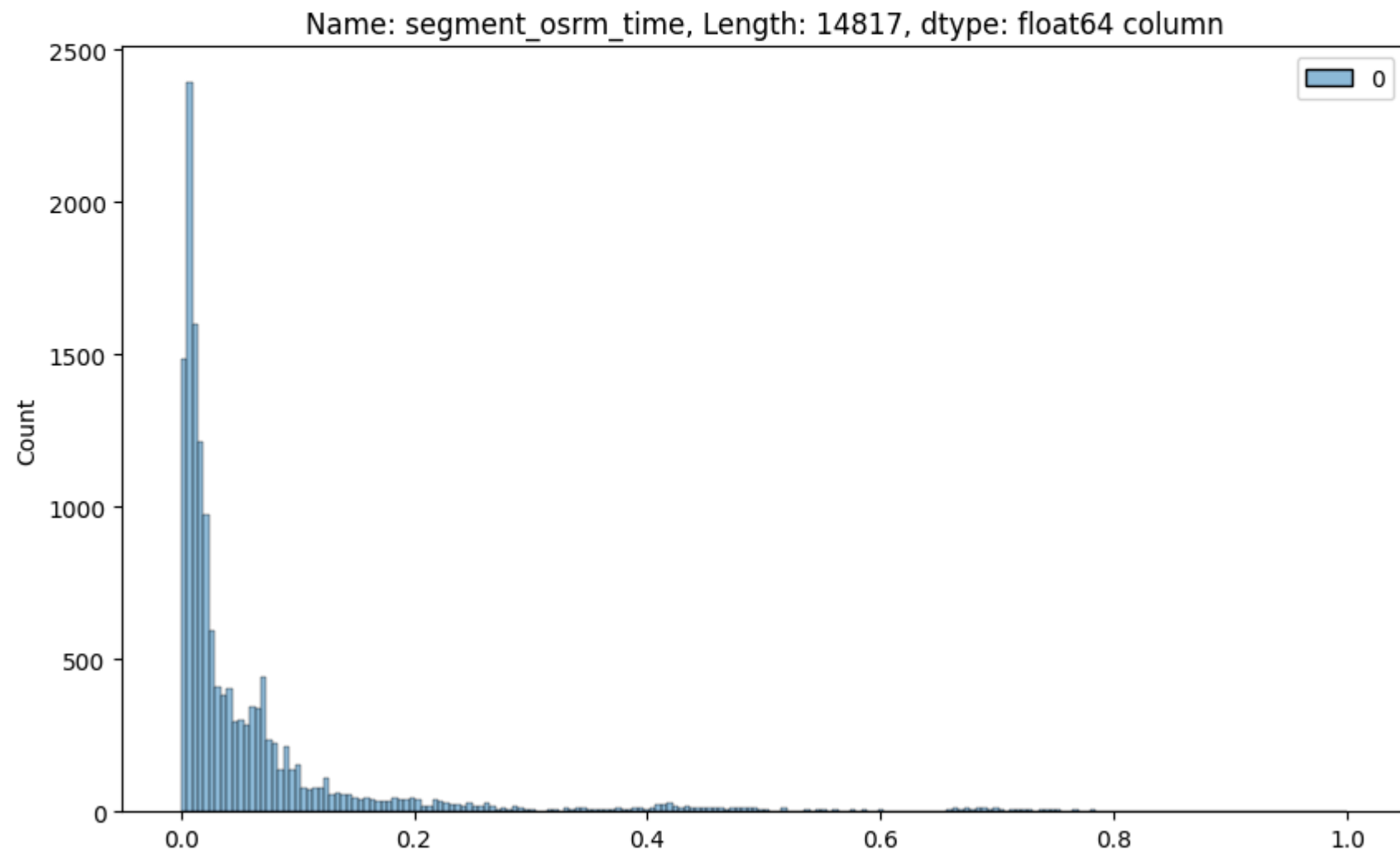
Normalized 0	1548.0
1	141.0
2	3308.0
3	59.0
4	340.0
	...
14812	82.0
14813	21.0
14814	281.0
14815	258.0
14816	274.0



```
In [ ]: plt.figure(figsize = (10, 6))
scaler = MinMaxScaler()
scaled = scaler.fit_transform(df2['segment_osrm_time'].to_numpy().reshape(-1, 1))
sns.histplot(scaled)
plt.title(f"Normalized {df2['segment_osrm_time']} column")
plt.plot()
```

Out[]: []

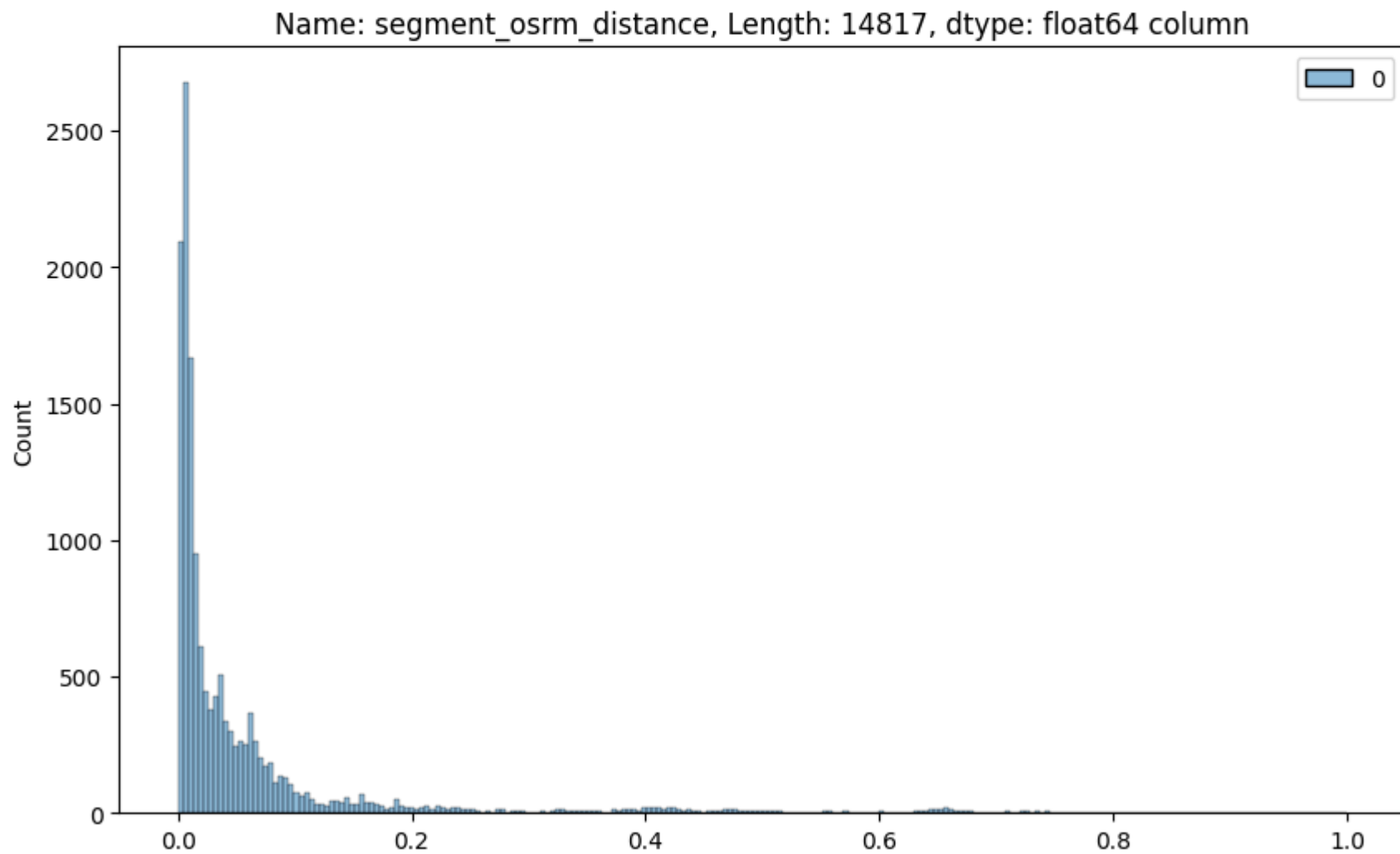
	Normalized 0	1008.0
1	65.0	
2	1941.0	
3	16.0	
4	115.0	
	...	
14812	62.0	
14813	11.0	
14814	88.0	
14815	221.0	
14816	67.0	



```
In [ ]: plt.figure(figsize = (10, 6))
scaler = MinMaxScaler()
scaled = scaler.fit_transform(df2['segment_osrm_distance'].to_numpy().reshape(-1, 1))
sns.histplot(scaled)
plt.title(f"Normalized {df2['segment_osrm_distance']} column")
plt.plot()
```

Out[]: []

Normalized 0	1320.4733
1	84.1894
2	2545.2678
3	19.8766
4	146.7919
	...
14812	64.8551
14813	16.0883
14814	104.8866
14815	223.5324
14816	80.5787



Column Standardization :

```
In [ ]: from sklearn.preprocessing import StandardScaler
```

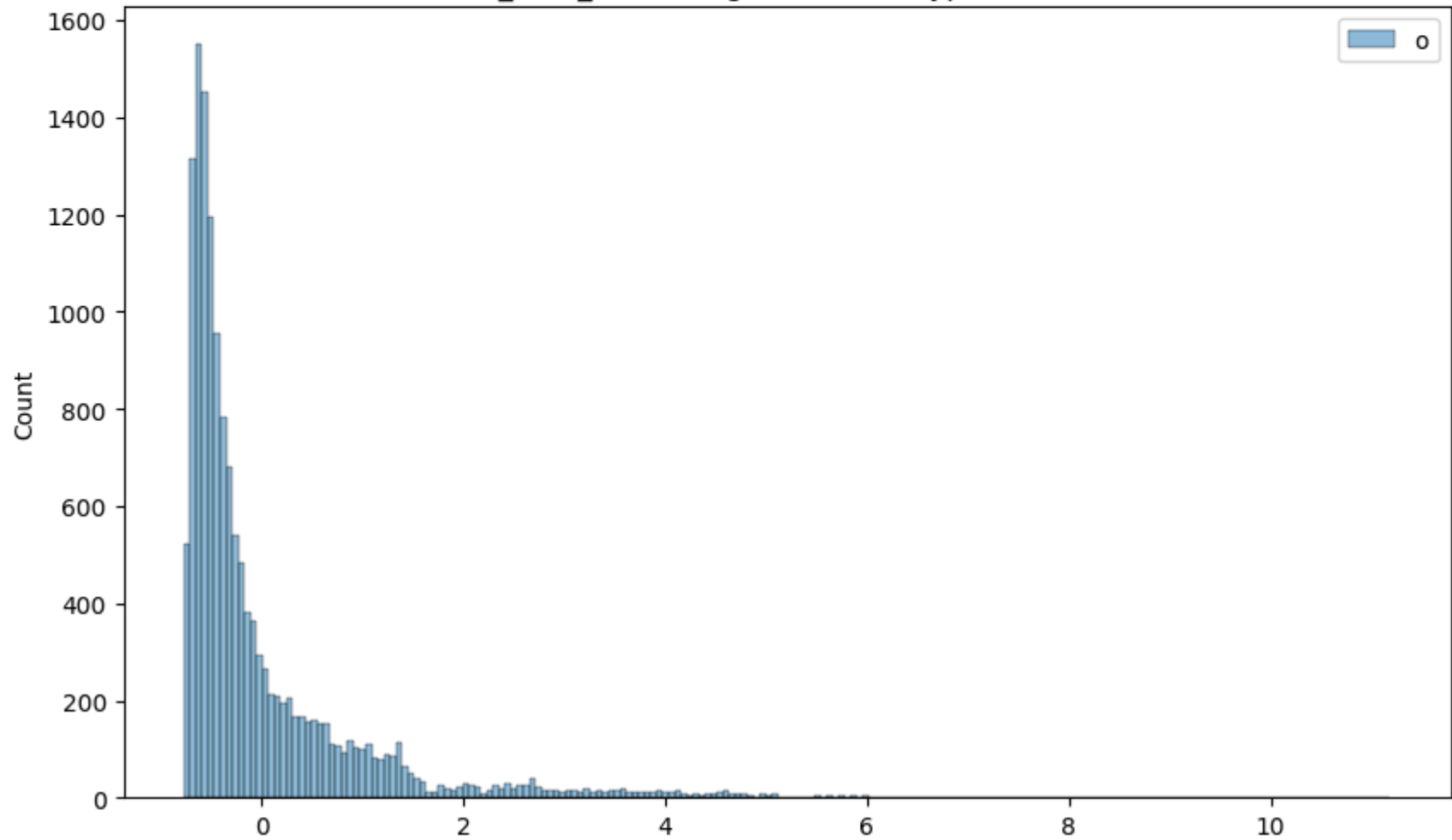
```
In [ ]: plt.figure(figsize = (10, 6))
# define standard scaler
scaler = StandardScaler()
# transform data
scaled = scaler.fit_transform(df2['od_total_time'].to_numpy().reshape(-1, 1))
sns.histplot(scaled)
plt.title(f"Standardized {df2['od_total_time']} column")
plt.legend('od_total_time')
plt.plot()
```

```
Out[ ]: []
```

Standardized 0	2260.11
1	181.61
2	3934.36
3	100.49
4	718.34

	...
14812	258.03
14813	60.59
14814	422.12
14815	348.52
14816	354.40

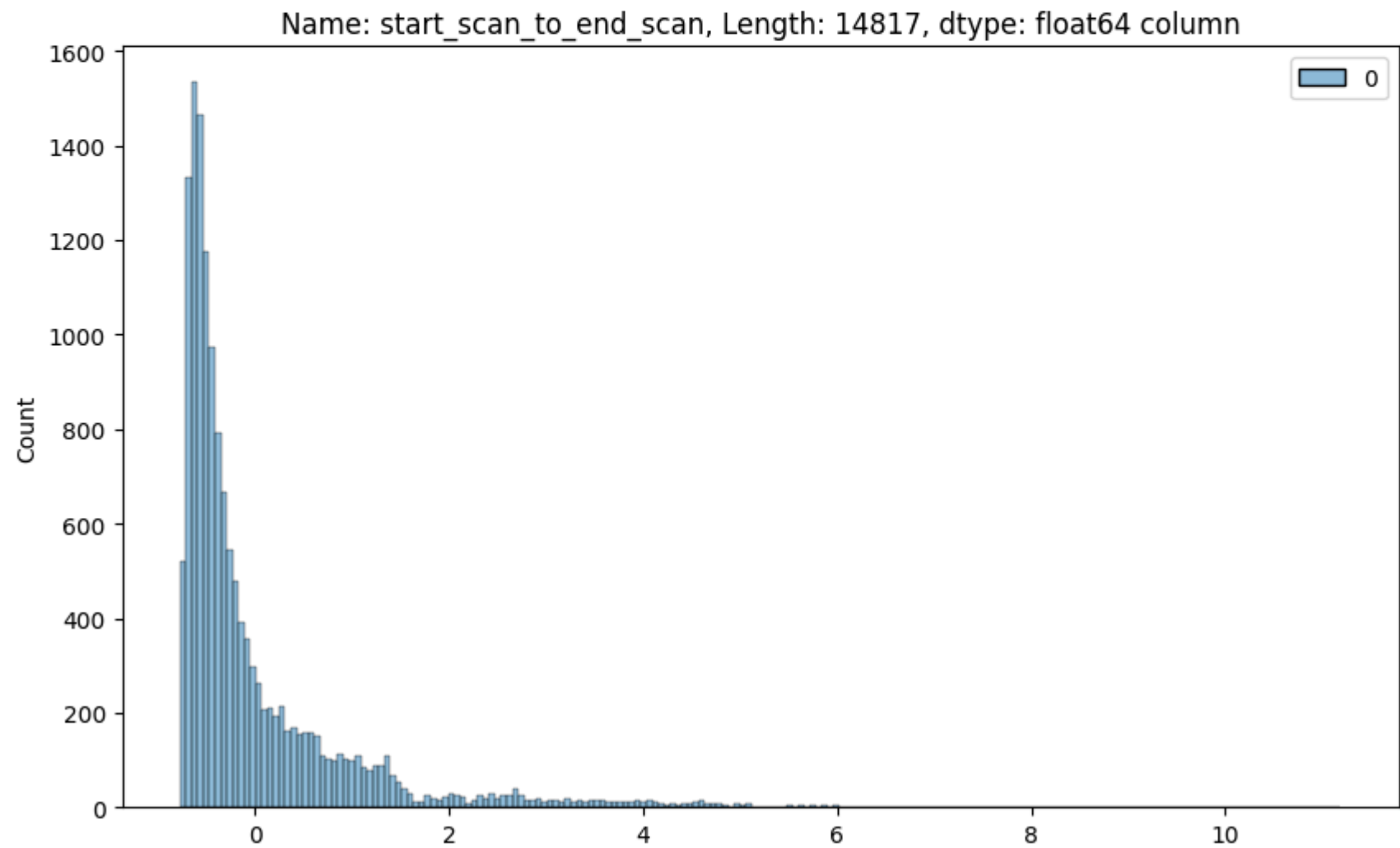
Name: od_total_time, Length: 14817, dtype: float64 column



```
In [ ]: plt.figure(figsize = (10, 6))
scaler = StandardScaler()
scaled = scaler.fit_transform(df2['start_scan_to_end_scan'].to_numpy().reshape(-1, 1))
sns.histplot(scaled)
plt.title(f"Standardized {df2['start_scan_to_end_scan']} column")
plt.plot()
```

Out[]: []

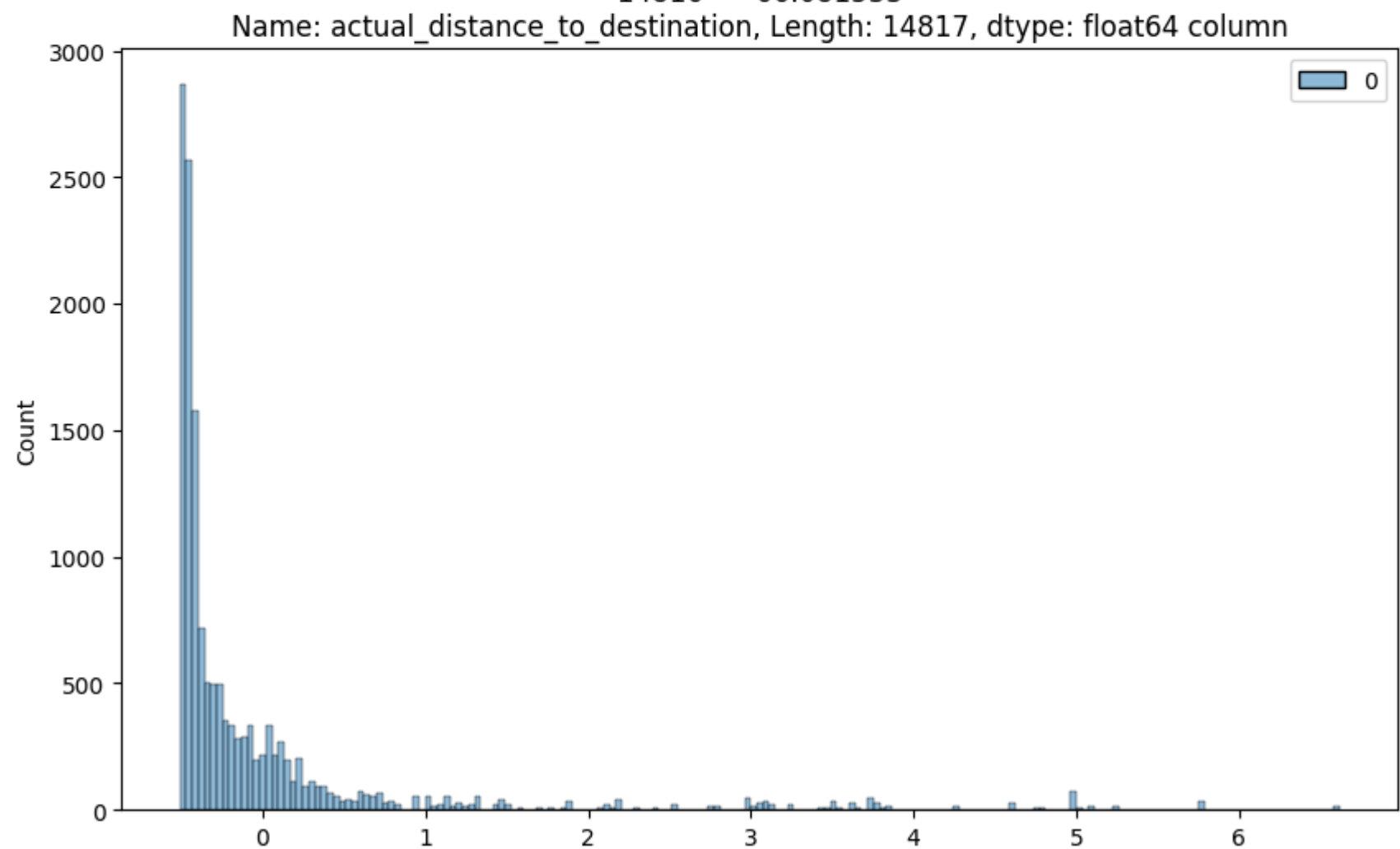
Standardized 0	2259.0
1	180.0
2	3933.0
3	100.0
4	717.0
	...
14812	257.0
14813	60.0
14814	421.0
14815	347.0
14816	353.0



```
In [ ]: plt.figure(figsize = (10, 6))
scaler = StandardScaler()
scaled = scaler.fit_transform(df2['actual_distance_to_destination'].to_numpy().reshape(-1, 1))
sns.histplot(scaled)
plt.title(f"Standardized {df2['actual_distance_to_destination']} column")
plt.plot()
```

Out[]: []

Standardized	0	824.732854
1	73.186911	
2	1927.404273	
3	17.175274	
4	127.448500	
	...	
14812	57.762332	
14813	15.513784	
14814	38.684839	
14815	134.723836	
14816	66.081533	

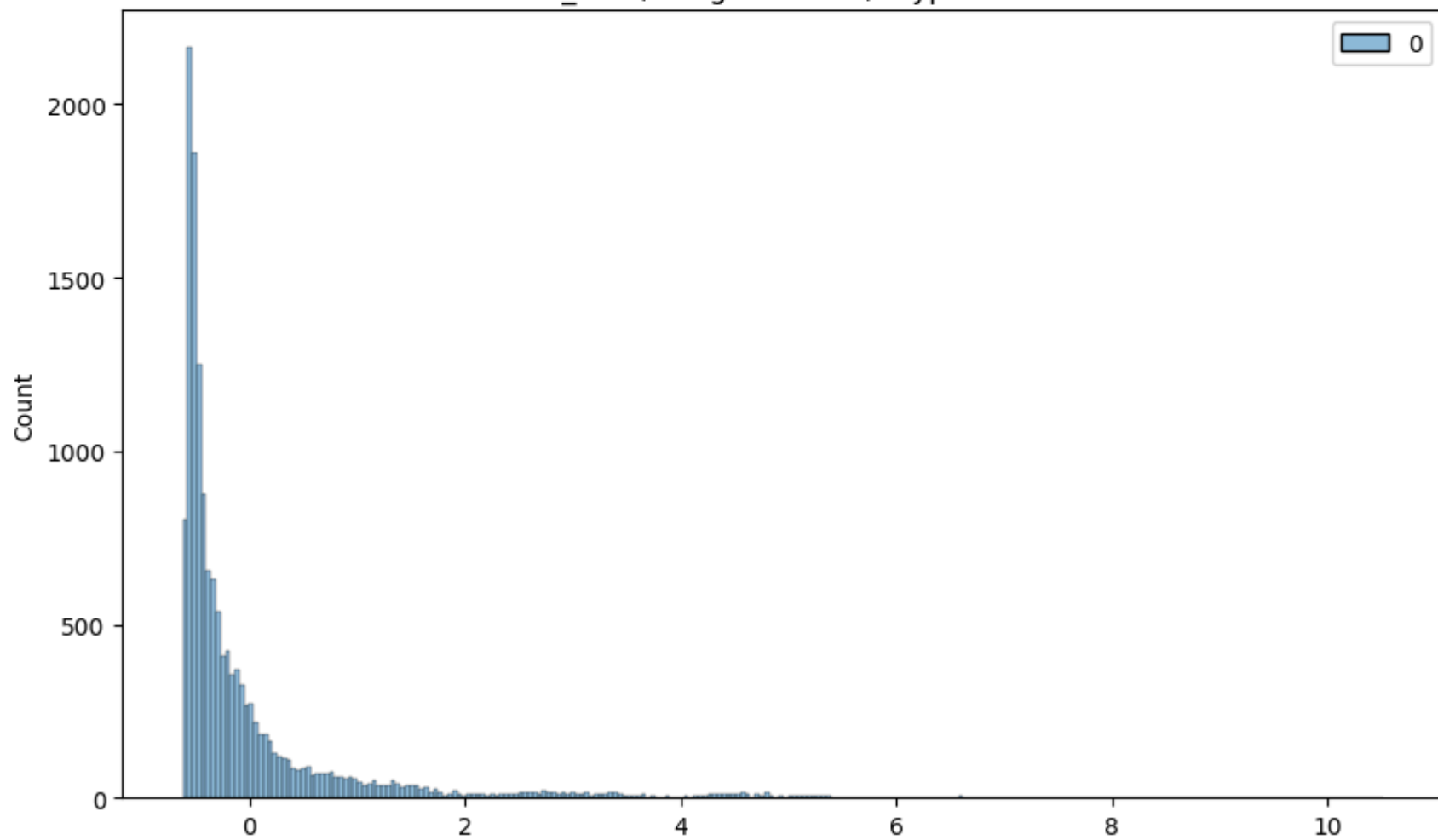


```
In [ ]: plt.figure(figsize = (10, 6))
scaler = StandardScaler()
scaled = scaler.fit_transform(df2['actual_time'].to_numpy().reshape(-1, 1))
sns.histplot(scaled)
plt.title(f"Standardized {df2['actual_time']} column")
plt.plot()
```

Out[]: []

Standardized 0	1562.0
1	143.0
2	3347.0
3	59.0
4	341.0
	...
14812	83.0
14813	21.0
14814	282.0
14815	264.0
14816	275.0

Name: actual_time, Length: 14817, dtype: float64 column



```
In [ ]: plt.figure(figsize = (10, 6))
scaler = StandardScaler()
scaled = scaler.fit_transform(df2['osrm_time'].to_numpy().reshape(-1, 1))
sns.histplot(scaled)
plt.title(f"Standardized {df2['osrm_time']} column")
plt.plot()
```

Out[]: []

Standardized 0 717.0

1 68.0

2 1740.0

3 15.0

4 117.0

...

14812 62.0

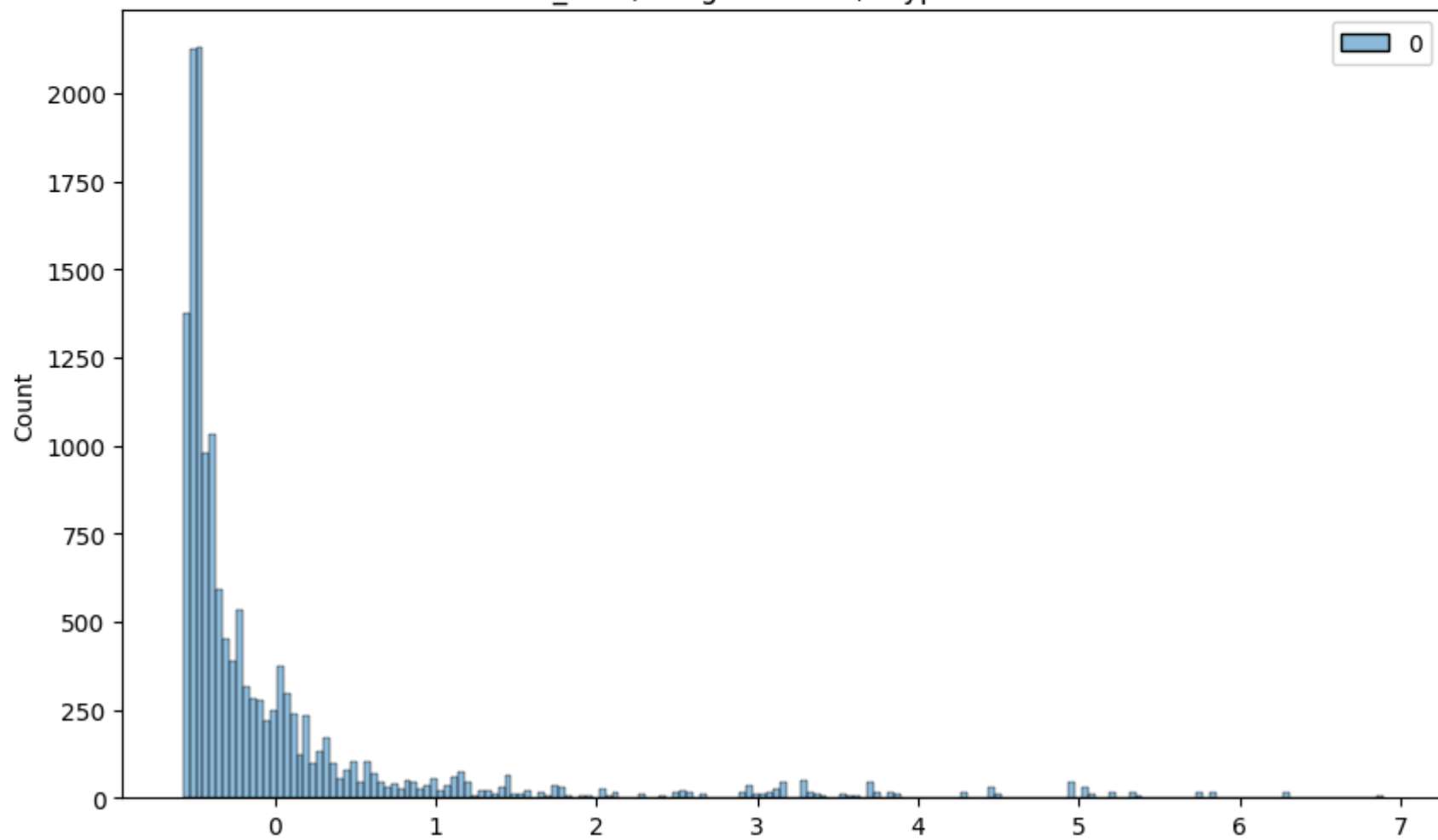
14813 12.0

14814 48.0

14815 179.0

14816 68.0

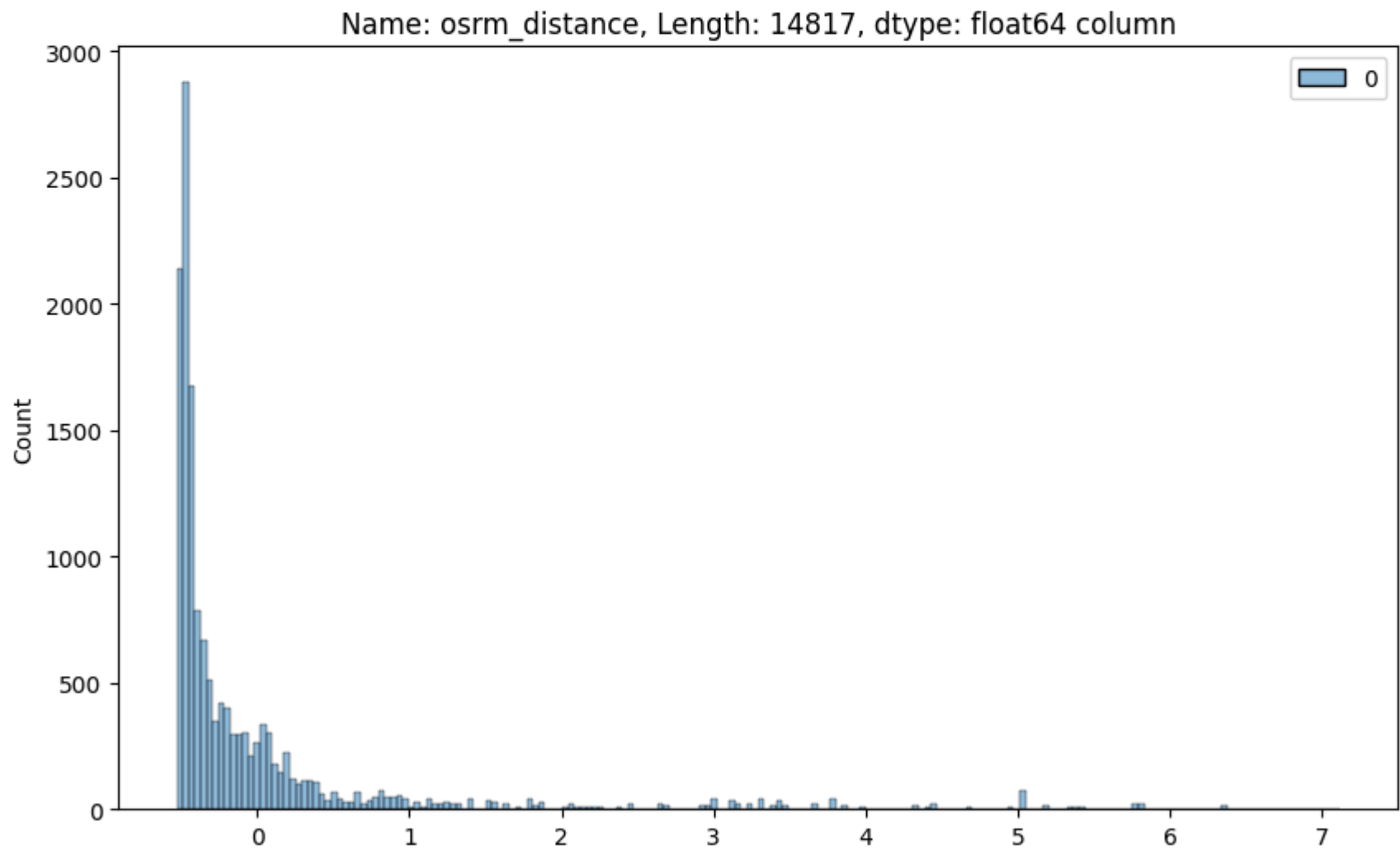
Name: osrm_time, Length: 14817, dtype: float64 column



```
In [ ]: plt.figure(figsize = (10, 6))
scaler = StandardScaler()
scaled = scaler.fit_transform(df2['osrm_distance'].to_numpy().reshape(-1, 1))
sns.histplot(scaled)
plt.title(f"Standardized {df2['osrm_distance']} column")
plt.plot()
```

Out[]: []

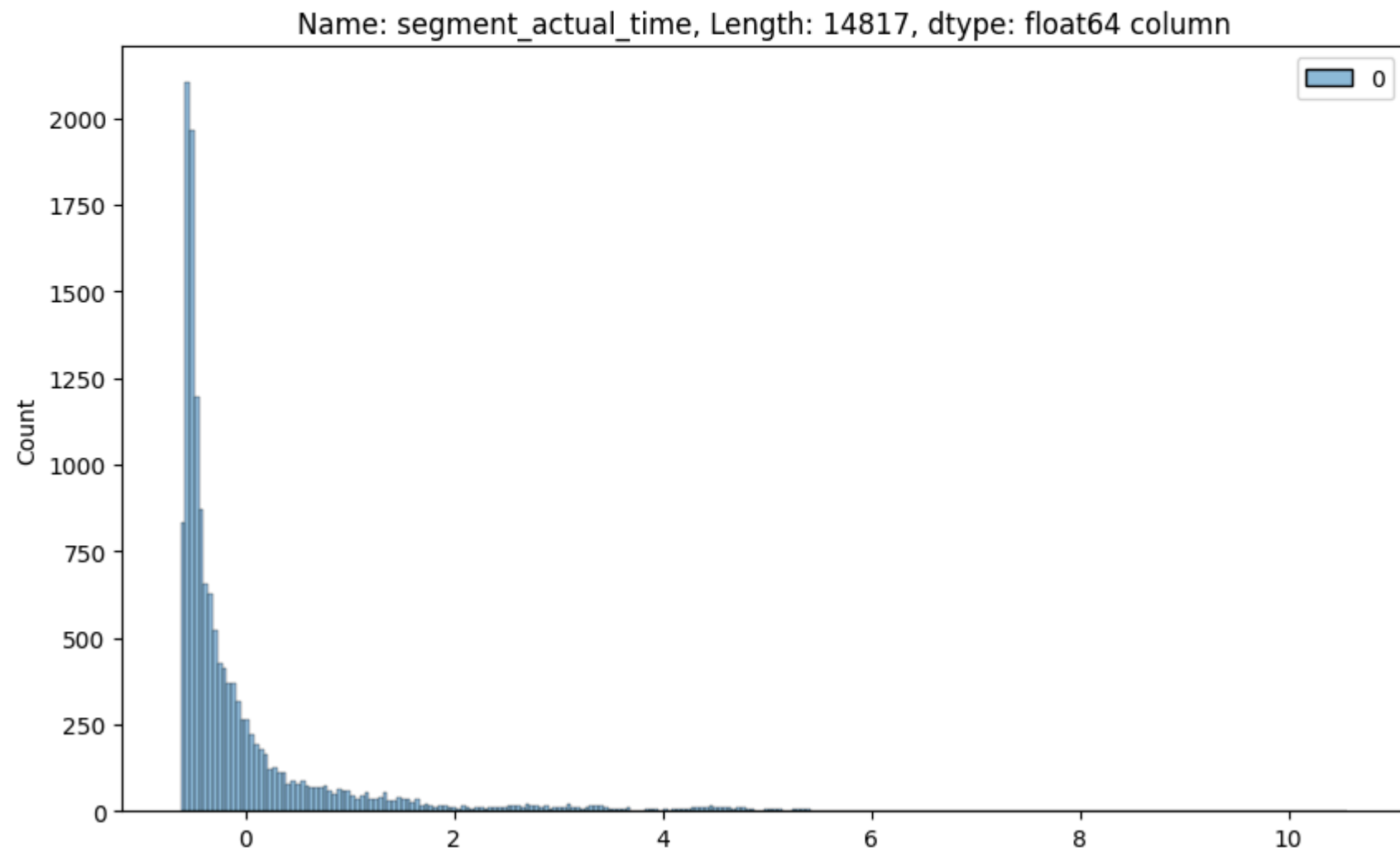
Standardized 0	991.3523
1	85.1110
2	2354.0665
3	19.6800
4	146.7918
	...
14812	73.4630
14813	16.0882
14814	58.9037
14815	171.1103
14816	80.5787



```
In [ ]: plt.figure(figsize = (10, 6))
scaler = StandardScaler()
scaled = scaler.fit_transform(df2['segment_actual_time'].to_numpy().reshape(-1, 1))
sns.histplot(scaled)
plt.title(f"Standardized {df2['segment_actual_time']} column")
plt.plot()
```

Out[]: []

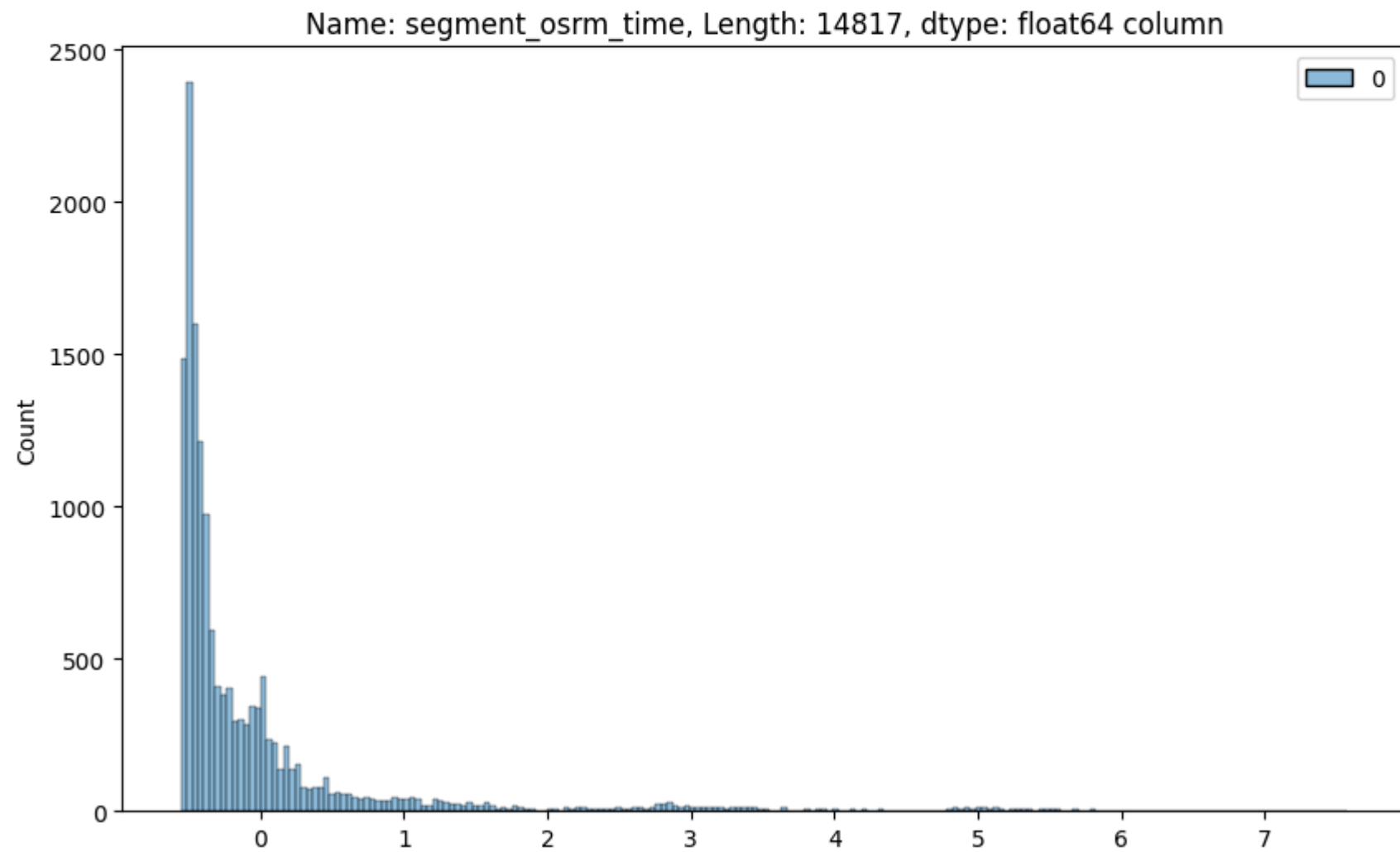
Standardized 0	1548.0
1	141.0
2	3308.0
3	59.0
4	340.0
	...
14812	82.0
14813	21.0
14814	281.0
14815	258.0
14816	274.0



```
In [ ]: plt.figure(figsize = (10, 6))
scaler = StandardScaler()
scaled = scaler.fit_transform(df2['segment_osrm_time'].to_numpy().reshape(-1, 1))
sns.histplot(scaled)
plt.title(f"Standardized {df2['segment_osrm_time']} column")
plt.plot()
```

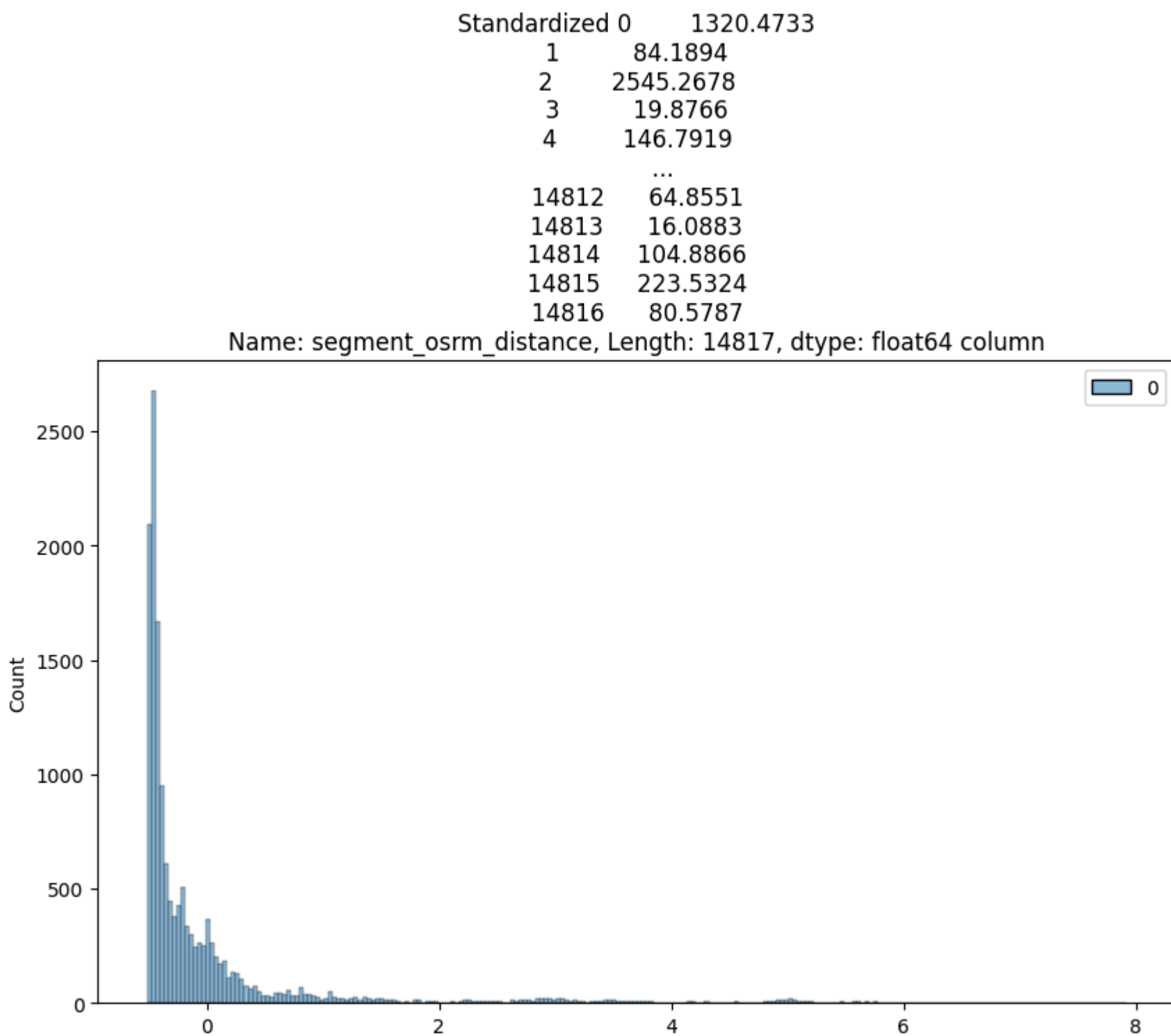
Out[]: []

	Standardized 0	1008.0
1	65.0	
2	1941.0	
3	16.0	
4	115.0	
	...	
14812	62.0	
14813	11.0	
14814	88.0	
14815	221.0	
14816	67.0	



```
In [ ]: plt.figure(figsize = (10, 6))
scaler = StandardScaler()
scaled = scaler.fit_transform(df2['segment_osrm_distance'].to_numpy().reshape(-1, 1))
sns.histplot(scaled)
plt.title(f"Standardized {df2['segment_osrm_distance']} column")
plt.plot()
```

Out[]: []



Business Insights :

- The data is given from the period '2018-09-12 00:00:16' to '2018-10-08 03:00:24'.
- There are about 14817 unique trip IDs, 1508 unique source centers, 1481 unique destination_centers, 690 unique source cities, 806 unique destination cities.
- Most of the data is for testing than for training.
- Most common route type is Carting.
- The names of 14 unique location ids are missing in the data.
- The number of trips start increasing after the noon, becomes maximum at 10 P.M and then start decreasing.
- Most orders come mid-month. That means customers usually make more orders in the mid of the month.
- Most orders are sourced from the states like Maharashtra, Karnataka, Haryana, Tamil Nadu, Telangana
- Maximum number of trips originated from Mumbai city followed by Gurgaon Delhi, Bengaluru and Bhiwandi. That means that the seller base is strong in these cities.
- Maximum number of trips ended in Maharashtra state followed by Karnataka, Haryana, Tamil Nadu and Uttar Pradesh. That means that the number of orders placed in these states is significantly high.
- Maximum number of trips ended in Mumbai city followed by Bengaluru, Gurgaon, Delhi and Chennai. That means that the number of orders placed in these cities is significantly high.
- Most orders in terms of destination are coming from cities like bengaluru, mumbai, gurgaon, bangalore, Delhi.
- Features start_scan_to_end_scan and od_total_time(created feature) are statistically similar.
- Features actual_time & osrm_time are statitically different.
- Features start_scan_to_end_scan and segment_actual_time are statistically similar.

- Features osrm_distance and segment_osrm_distance are statistically different from each other.
- Both the osrm_time & segment_osrm_time are not statistically same.

Recommendations :

- The OSRM trip planning system needs to be improved. Discrepancies need to be catered to for transporters, if the routing engine is configured for optimum results.
- The osrm distance and actual distance covered are also not same i.e. maybe the delivery person is not following the predefined route which may lead to late deliveries or the osrm devices is not properly predicting the route based on distance, traffic and other factors. Team needs to look into it.
- Most of the orders are coming from/reaching to states like Maharashtra, Karnataka, Haryana and Tamil Nadu. The existing corridors can be further enhanced to improve the penetration in these areas.
- Customer profiling of the customers belonging to the states Maharashtra, Karnataka, Haryana, Tamil Nadu and Uttar Pradesh has to be done to get to know why major orders are coming from these atates and to improve customers' buying and delivery experience.

In [191...

In []: